Proiectarea Algoritmilor

Curs 7 – Componente tare conexe, Puncte de articulație, Punți



Bibliografie curs 7

- Giumale – Introducere in Analiza Algoritmilor cap. 5.2 și 5.3
- Cormen – Introducere în Algoritmi cap. Algoritmi elementari de grafuri (23) – Componente Tare Conexe
- http://en.wikipedia.org/wiki/Tarjan%27s_strongly_connected_components_algorithm

Obiective

- “Descoperirea” algoritmilor de:
  - Identificare a componentelor tare conexe;
  - Identificare a punctelor de articulație;
  - Identificare a punților.
- Identificarea structurilor de date necesare pentru reducerea complexității acestor algoritmi.



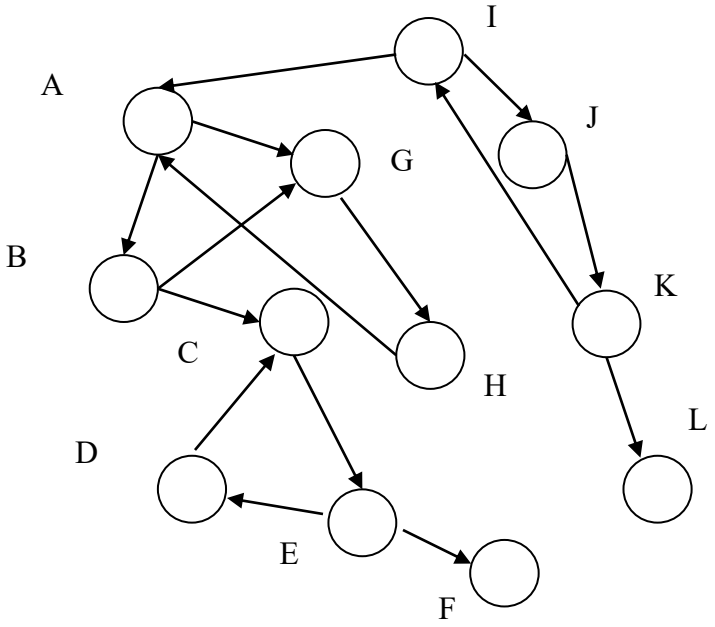
Componente Tare Conexe (CTC)

- Definiție: Fie G = (V,E) un graf orientat. G este tare-conex $\Leftrightarrow \forall$ u,v$\in$V $\exists$ o cale u..v și o cale v..u (u$\in$R(v) și v$\in$R(u)).

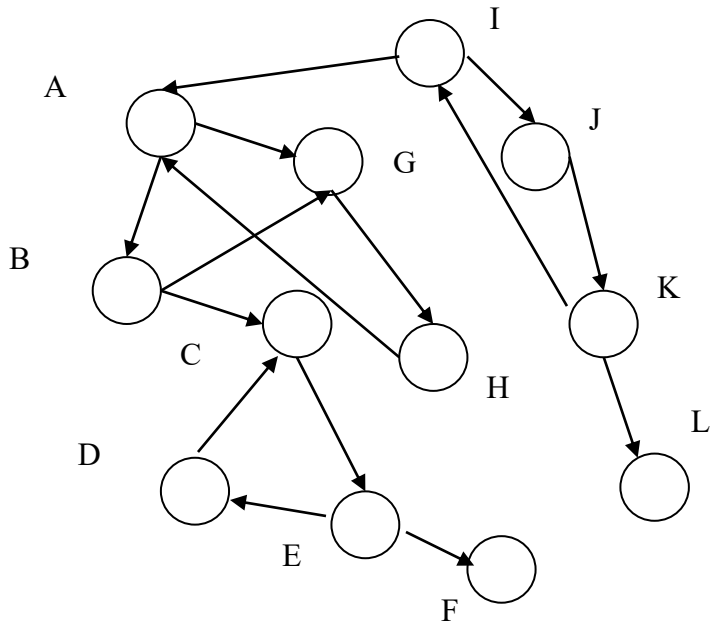
- Definiție: G = (V,E) graf orientat. G' = (V',E'), V'$\subseteq$V, E'$\subseteq$E. G' este o CTC a lui G $\Leftrightarrow$ G' e tare-conex ($\forall$ u,v$\in$V', u$\in$R(v) și v$\in$R(u)) și G' este maximal (ca număr de noduri).

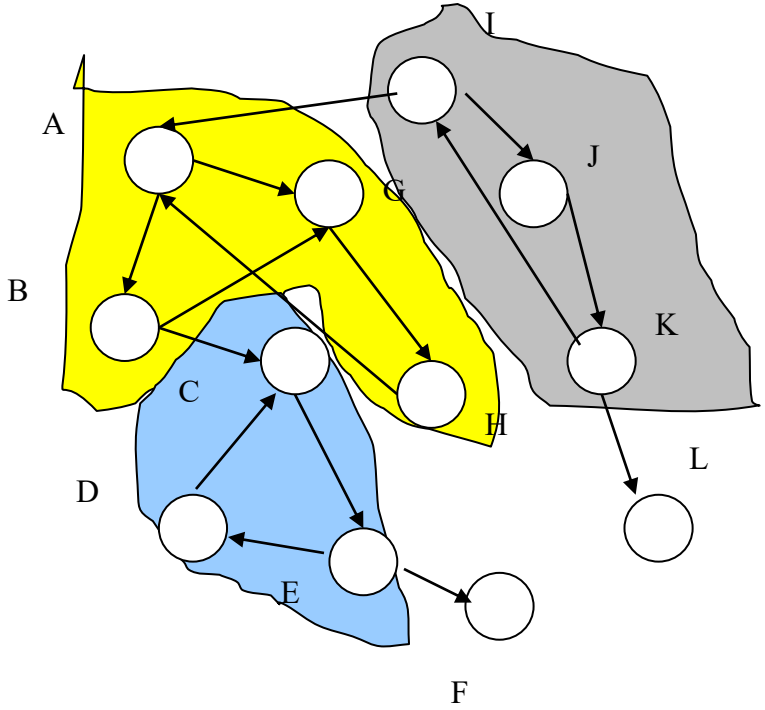
- Lema 5.6: G = (V,E) graf orientat, G' CTC => $\forall$ u,v$\in$V' avem $\forall$ u..v din G are noduri exclusiv in V'
  - Dem: $\forall$ z a.î. u..z..v => z$\in$R(u) și v$\in$R(z). Dar u$\in$R(v) => z$\in$R(v) => v și z sunt în aceeași CTC.

Exemplu (I) – determinare CTC



Exemplu (II) – determinare CTC(2)



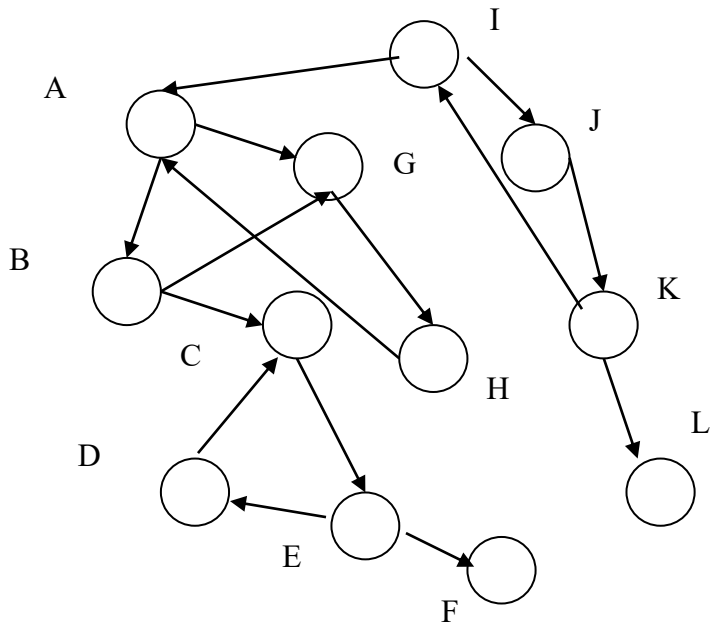


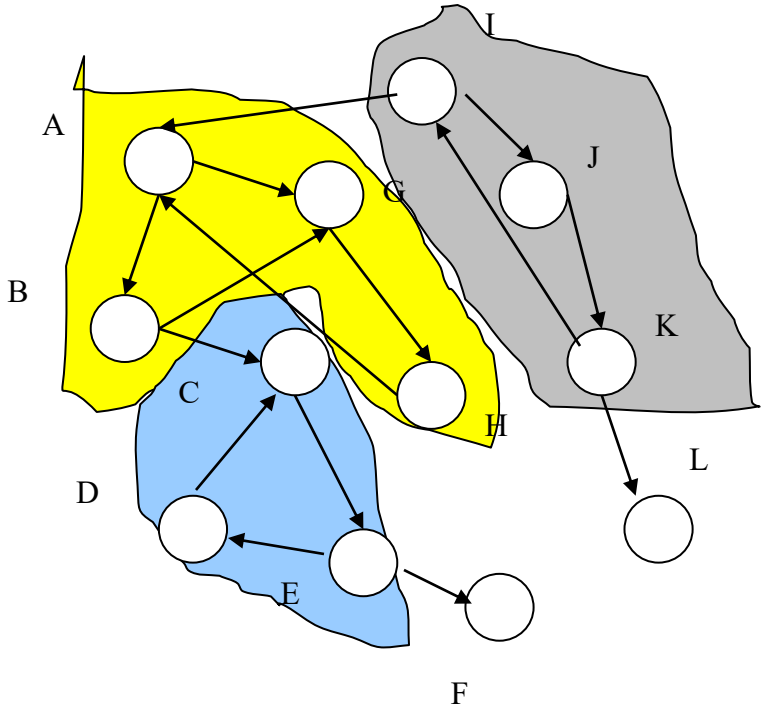
Componente Tare Conexe (CTC)

- Teorema 5.10: G = (V,E) orientat, G' = (V',E') o CTC a lui G. Toate nodurile $v \in V'$ sunt grupate in același Arb(u) construit de DFS(G), unde u este primul nod descoperit al componentei.
  - Dem: $\forall\ v \in V'$, $v \neq u$, $\exists$ u..v drum cu noduri albe la momentul descoperirii lui u (d(u)) + toate nodurile drumului sunt în V' (conf. Lema 5.6) => (din Teorema drumurilor albe) v este descendent al lui u în Arb(u)

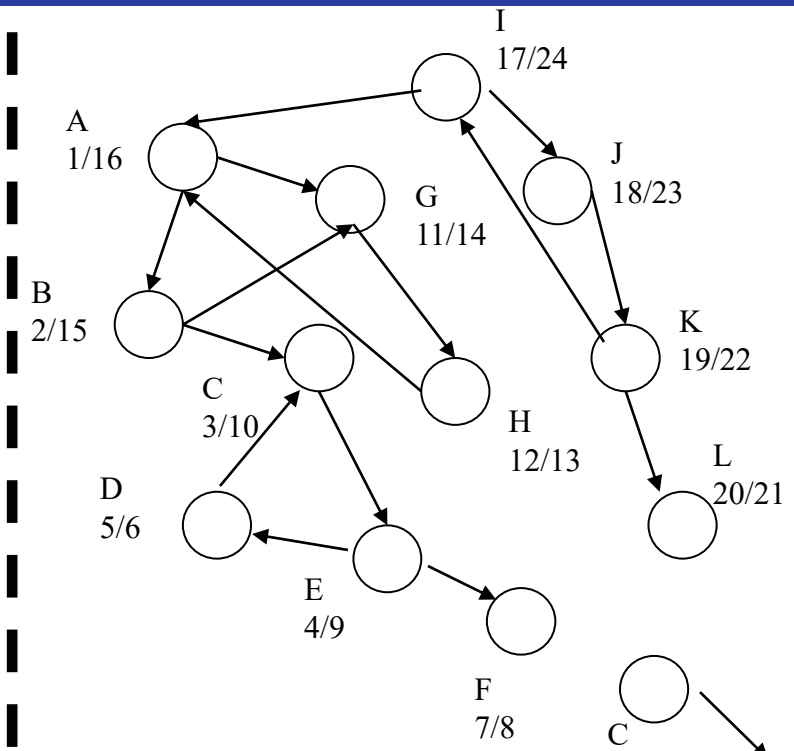
Exemplu (III) – DFS

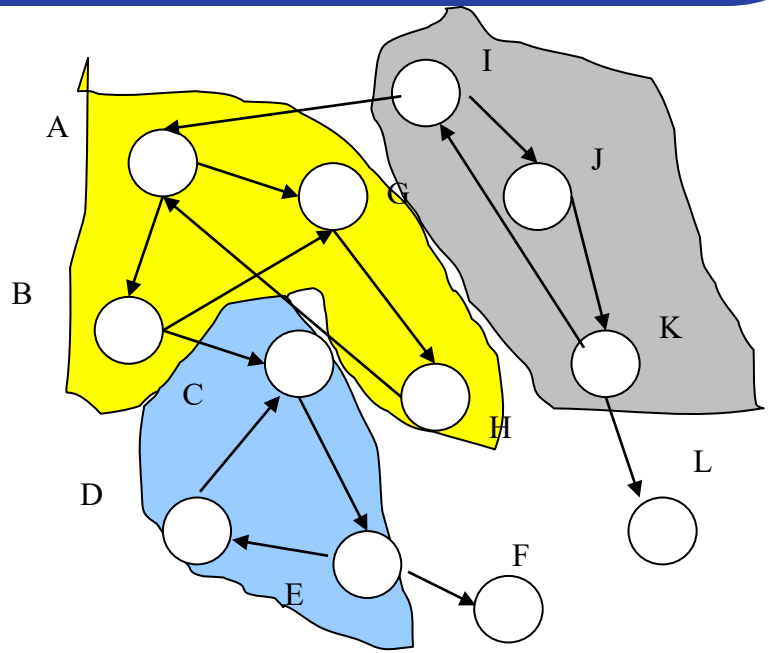
- Aplicare DFS pornind din primul nod al fiecărei CTC



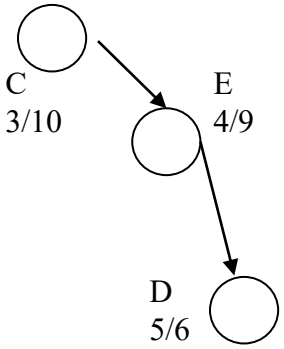


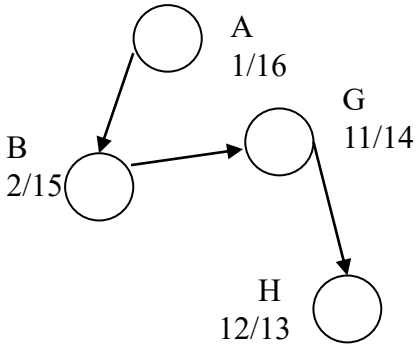
Exemplu (IV) - DFS(2)

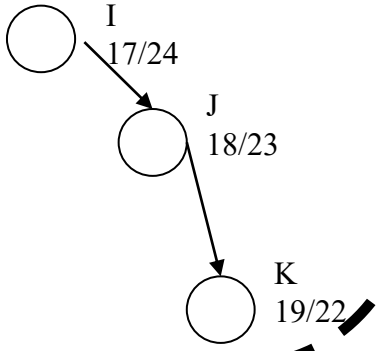




Conform Teoremei nodurile din aceeași CTC sunt grupate în același arbore DFS!







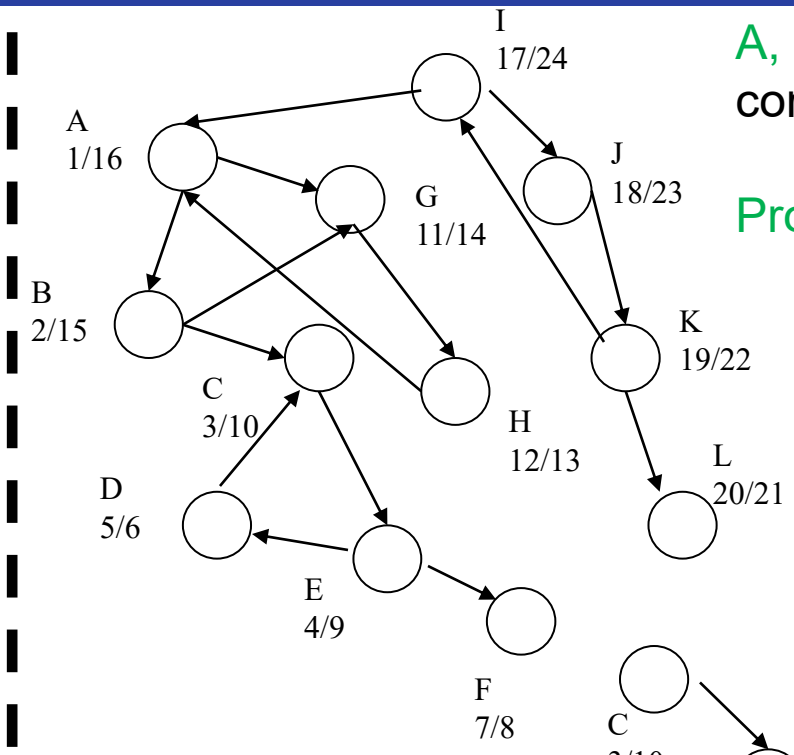
Componente Tare Conexe (CTC)

- Definiție: G = (V,E) orientat, $u \in V$. $\Phi(u)$ = strămoș DFS al lui u determinat în cursul DFS(G) dacă:
  - $\Phi(u) \in R(u)$
  - $f(\Phi(u)) = \max\{f(v) \mid v \in R(u)\}$

- Ce e $\Phi(u)$? $\Phi(u)$ este primul nod din CTC descoperit de DFS(G)

- Teorema 5.11: $\Phi(u)$ satisface următoarele proprietăți:
  1. $f(u) \leq f(\Phi(u))$ când e egalitate? u este primul nod din CTC
  2. $\forall v \in R(u), f(\Phi(v)) \leq f(\Phi(u))$ ce înseamnă ca e egalitate? u si v sunt in aceeași CTC
  3. $\Phi(\Phi(u)) = \Phi(u)$ Dem: $\Phi(u) \in R(u) \xrightarrow{2} f(\Phi(\Phi(u))) \leq f(\Phi(u))$; $\xrightarrow{1} f(\Phi(\Phi(u))) \geq f(\Phi(u)) \rightarrow f(\Phi(\Phi(u))) = f(\Phi(u)) \rightarrow \Phi(\Phi(u)) = \Phi(u)$

Exemplu (V) – stramosi



A, C, I sunt strămoși DFS ai nodurilor din componenta conexă din care fac parte.

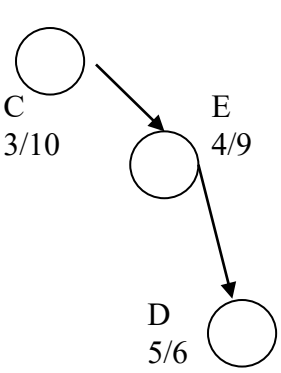
Proprietățile strămoșilor:

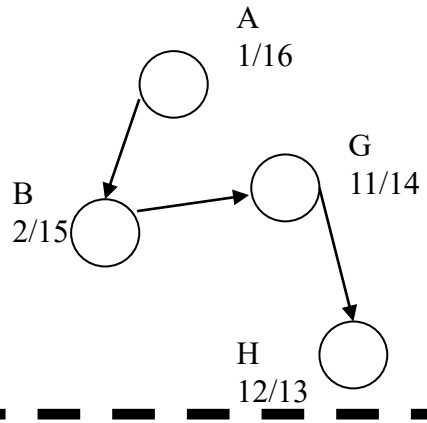
- $f(u) \leq f(\Phi(u))$

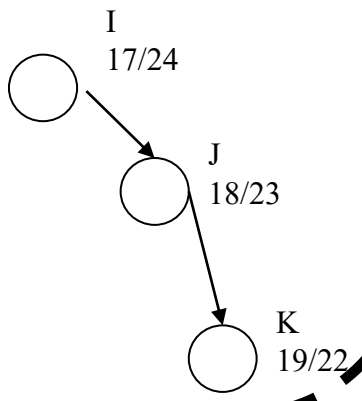
  Ex: $f(B) \leq f(\Phi(B)) = f(A)$
- $\forall\ v \in R(u),\ f(\Phi(v)) \leq f(\Phi(u))$

  Ex: $E \in R(B),\ f(\Phi(E)) = f(C) \leq f(A) = f(\Phi(B))$
- $\Phi(\Phi(u)) = \Phi(u)$

  Ex: $\Phi\ (\Phi(E)) = \Phi(C) = C = \Phi(E)$







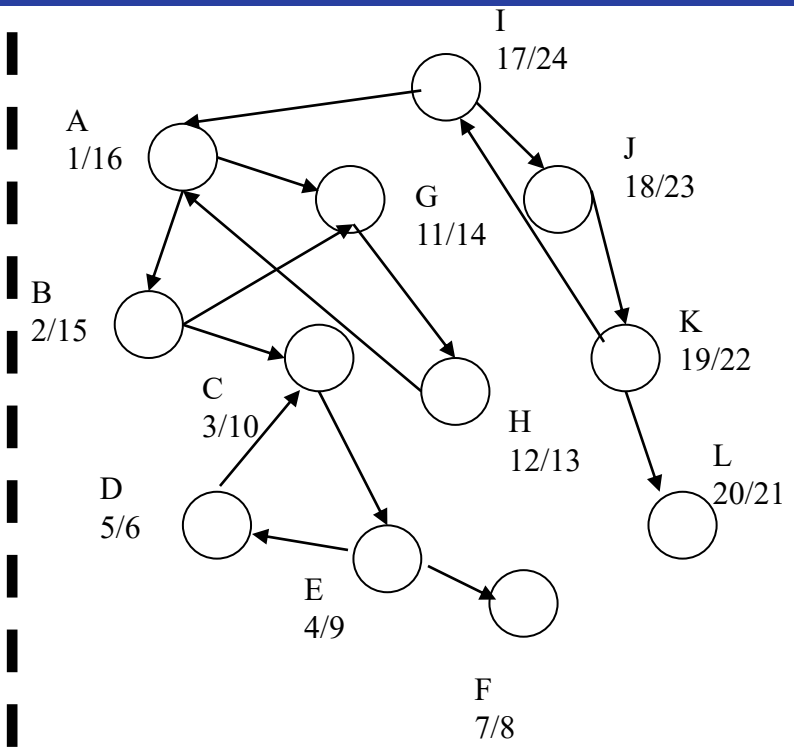
Componente Tare Conexe (CTC)

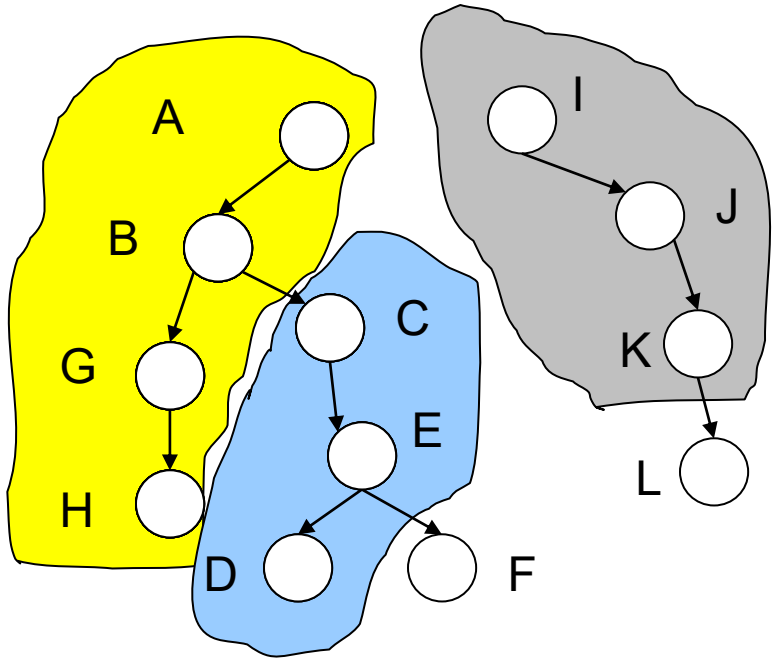
- Din Definiție: $\Phi(u) \in R(u)$ și $f(\Phi(u)) = \max\{f(v) \mid v \in R(u)\}$.

- Teorema 5.12. G = (V,E) orientat, $\forall\ u \in V$, u este descendent al lui $\Phi(u)$ în Arb($\Phi(u)$) construit de DFS.
  - Dem: prin considerarea tuturor culorilor posibile ale lui $\Phi(u)$ la momentul d(u).

- Teorema 5.13. G = (V,E) orientat, $\forall\ u,v \in V$; u și v aparțin aceleiași CTC $\Leftrightarrow$ $\Phi(u) = \Phi(v)$.
  - Dem folosind proprietățile strămoșilor :
    - $\forall\ u,v \in$ aceleiași CTC => $\Phi(u) = \Phi(v)$: $v \in R(u)$, $f(\Phi(v)) \leq f(\Phi(u))$ și $u \in R(v)$, $f(\Phi(u)) \leq f(\Phi(v))$ => $f(\Phi(u)) = f(\Phi(v))$ → $\Phi(u) = \Phi(v)$
    - $\Phi(u) = \Phi(v)$ => $\Phi(u) \in R(u)$ => u și $\Phi(u) \in$ aceleiași CTC
      și $\Phi(u) \in R(v)$ => v și $\Phi(u) \in$ aceleiași CTC
      => u și v $\in$ aceleiași CTC

Exemplu (VI)





Primul nod dintr-o CTC descoperit prin DFS va avea drept succesori în arborele generat de DFS toate elementele componentei conexe!

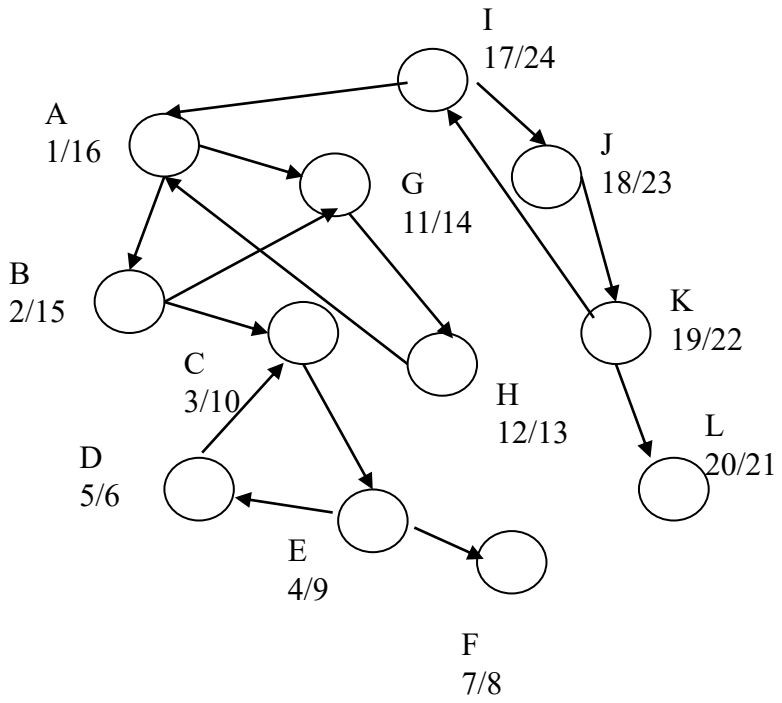
Componente Tare Conexe (CTC)

- Probleme:
  - vrem ca fiecare arbore construit să conțină o CTC.
  - trebuie să eliminăm nodurile care nu sunt în componenta conexă.

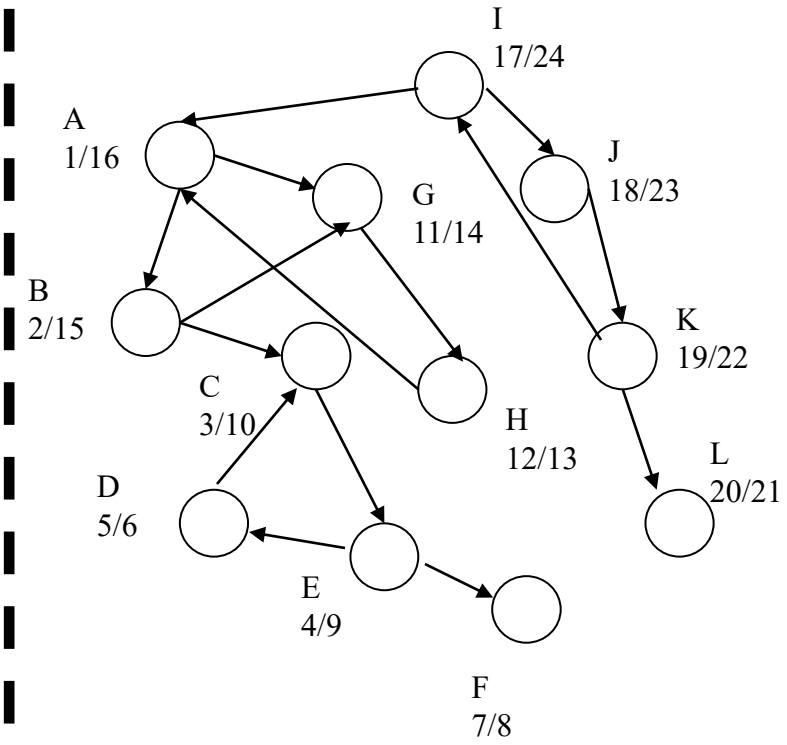
- Idee – eliminăm nodurile ce nu aparțin CTC! Cum?
  - Dacă aparțin Arb(u) și nu CTC => ∃ u..v și ∄ v..u.
  - → DFS pe graful transpus, în ordinea descrescătoare a timpilor de finalizare obținuți din DFS pe graful normal!

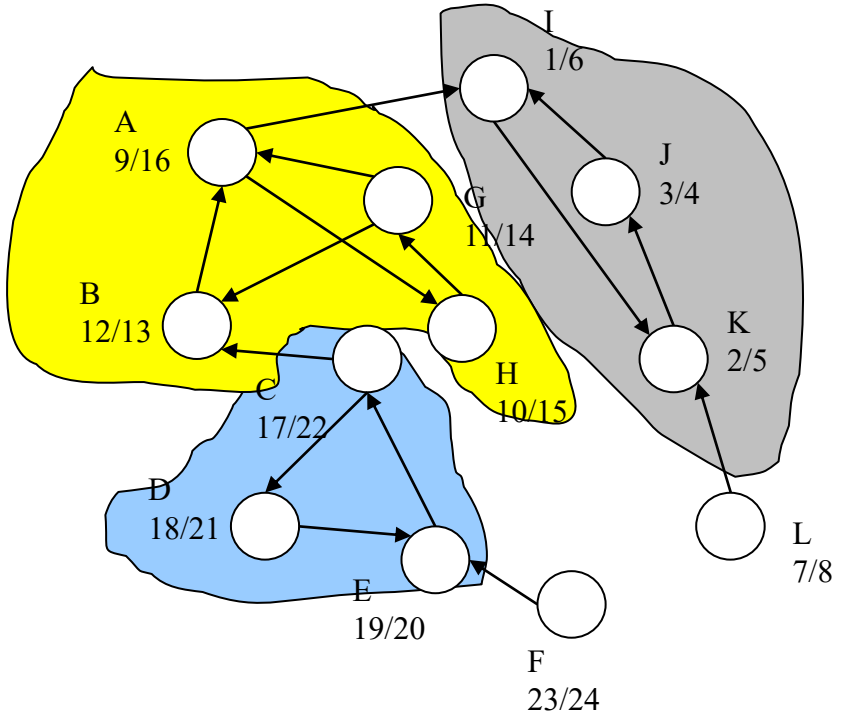


Exemplu (VII) – DFS (G^T)



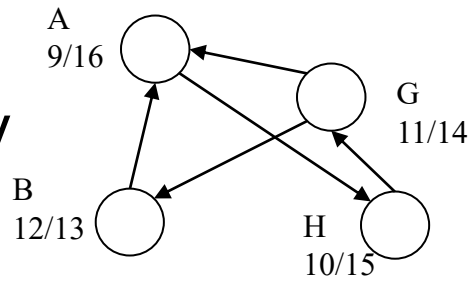
Exemplu (VIII) – DFS (G^T) (2)

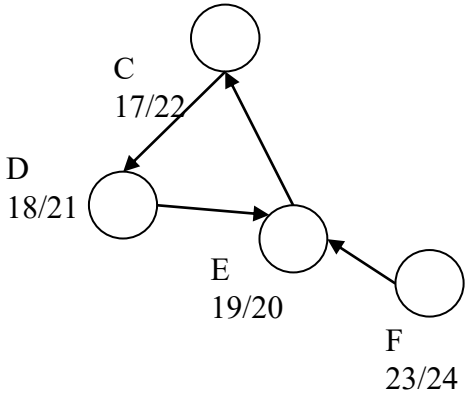


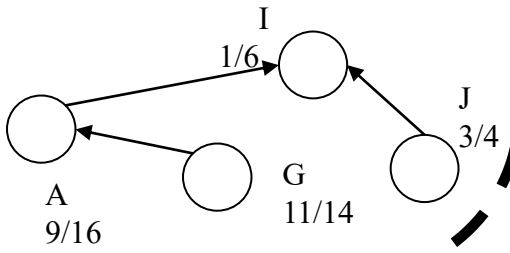


Componente Tare Conexe (CTC)

- Cazuri în DFS(G^T):
  - v este în CTC descoperita din u → v poate fi descoperit din u și în DFS(G^T).
  - v∉CTC dar v∈Arb(u) în DFS(G)→ nu va fi atins în DFS(G^T) din u.
  - v∉CTC dar ∃ v..u în G → f(v) > f(u) → v va fi deja colorat în negru când se explorează u.

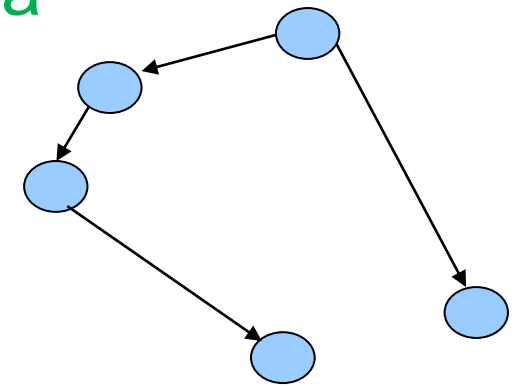






Observatii

- Înlocuind componentele tare conexe cu noduri obținem un graf aciclic. De ce?
- Pentru că altfel am avea o singură CTC!



- Prima parcurgere DFS este o sortare topologică. De ce?
- Pentru că sortează nodurile în ordinea inversă a timpilor de finalizare a strămoșilor fiecărei CTC!

Pseudocod algoritm CTC

- Algoritmul lui Kosaraju:

- CTC(G)
  - DFS(G)
  - G^T = transpune(G)
  - DFS(G^T) (în bucla principală se tratează nodurile în ordinea descrescătoare a timpilor de finalizare de la primul DFS)

- Componentele conexe sunt reprezentate de pădurea de arbori generați de DFS(G^T).

Complexitate?

Algoritmul lui Kosaraju

Complexitate
O(n+m)

n = numar noduri

m = numar muchii



Corectitudine algoritm CTC (1)

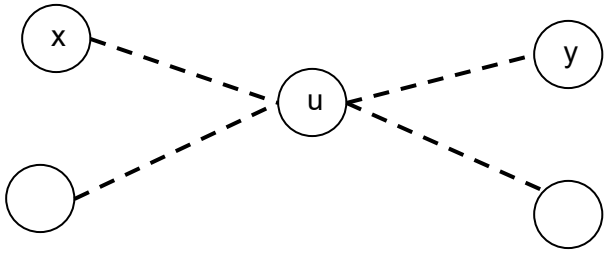
- Teoremă: Algoritmul CTC calculează corect componentele tare conexe ale unui graf G = (V,E).
- Dem. prin inducție după nr. de arbori de adâncime găsiți de DFS al G^T că vârfurile din fiecare arbore formează o CTC:
  - Fiecare pas demonstrează că arborele format în acel pas e o CTC, presupunând că toți arborii produși deja sunt CTC.
  - P_1: trivial pentru că ∄ arbori anteriori.
  - $P_n \rightarrow P_{n+1}$: Fie arborele T obținut in pasul curent având rădăcina r. Notăm $C_r = \{v \in V \mid \Phi(v) = r\}$.

Corectitudine algoritm CTC (2)

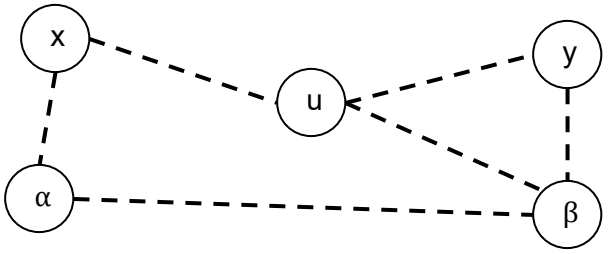
- Demonstrăm că $u \in T \Leftrightarrow u \in C_r$:
- $u \in C_r \rightarrow u \in T$:
  - $u \in C_r \rightarrow \exists$ r..u → toate nodurile din C_r ajung în același arbore DFS (Arb(r)). Dar $r \in C_r$ și r e rădăcina lui T → $\forall u \in C_r$ => $u \in T$
- $u \in T \rightarrow u \in C_r$: demonstrăm că $\forall$ w a.î. $f(\Phi(w)) > f(r)$ sau $f(\Phi(w)) < f(r)$, $w \notin T$
  - Dacă $f(\Phi(w)) > f(r)$ → la d(r), w e deja pus în CTC cu rădăcina $\Phi(w)$ pt. că nodurile sunt considerate în ordinea inversă a timpilor de finalizare → $w \notin T$
  - Dacă $f(\Phi(w)) < f(r)$ → $w \notin T$ pt. că altfel ($w \in T$) → $\exists$ r..w în G^T → $\exists$ w..r în G → $r \in R(w)$, → $f(\Phi(w)) \geq f(\Phi(r)) = f(r)$ (F)
  - → T conține doar nodurile pt. care $\Phi(w) = r$ → $T = C_r$

Puncte de articulație. Def. Exemple

- Definiție: G = (V,E) graf neorientat, u∈V. U este punct de articulație dacă ∃ x,y∈V, x ≠ y, x ≠ u, y ≠ u, a.î. ∀ x..y în G trece prin u.



Orice drum x..y trece prin u → u este punct de articulație.



Exista x..α..y care nu trece prin u → u nu mai este punct de articulație!

Algoritm naiv de detectare a punctelor de articulație

- Elimină fiecare nod și verifică conectivitatea grafului rezultat:
  - Graf conex → nodul nu e punct de articulație.
  - Altfel → punct de articulație.

Complexitate?

O(V(V+E))

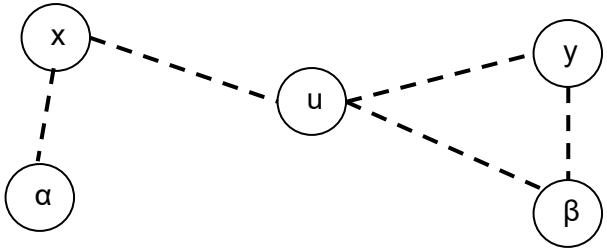


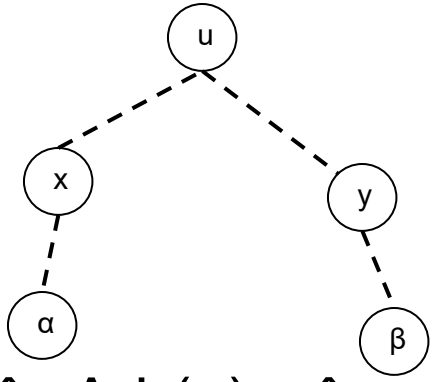
Puncte de articulație. Teoremă

- Teorema 5.15: G = (V,E), graf neorientat și $u \in V$. U este punct de articulație în G $\Leftrightarrow$ în urma DFS în G una din proprietățile de mai jos este satisfăcută:
  - p(u) = null și u domină cel puțin 2 subarbori;
  - p(u) ≠ null și $\exists v$ descendent al lui u în Arb(u) a.î. $\forall x \in$ Arb(v) și $\forall$(x,z) parcursă de DFS(G) avem d(z) ≥ d(u).

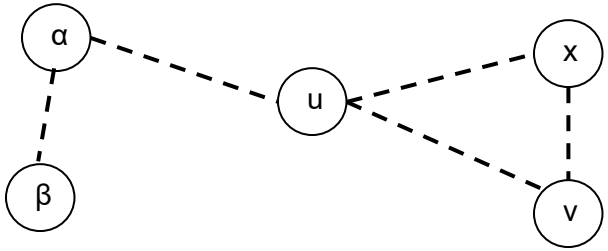
Situații posibile

- 1) p(u) = null și u domina cel puțin 2 subarbori:

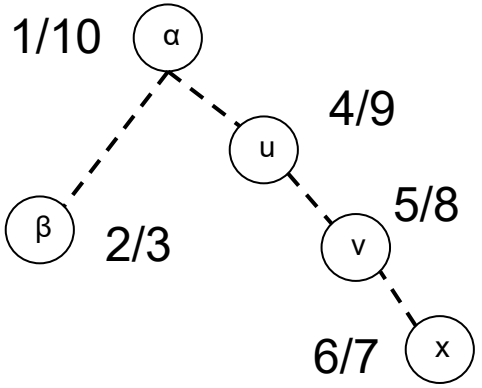




- 2) p(u) ≠ null și ∃v descendent al lui u în Arb(u) a.î. ∀x∈Arb(v) și ∀(x,z) parcursă de DFS(G) d(z) ≥ d(u):

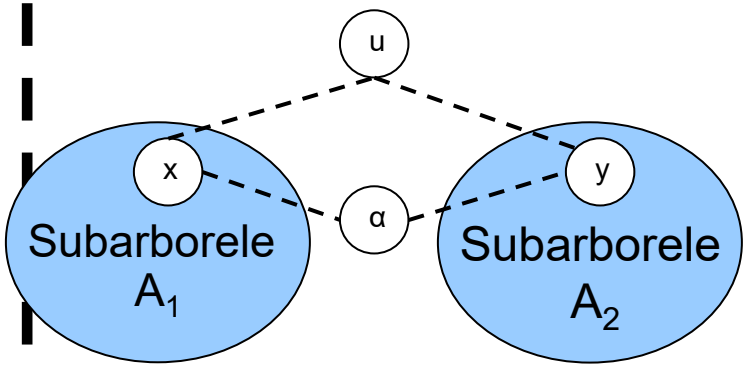


Pentru orice nod din subarborele lui v nu există nici o muchie înapoi spre u sau spre un nod descoperit înaintea lui u.



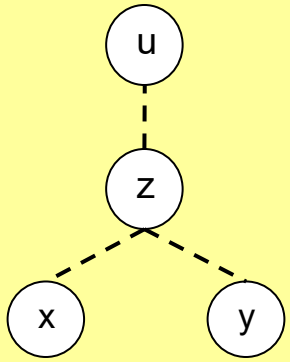
Puncte de articulație. Demonstrație teoremă (Ia)

- p(u) = null și u domină cel puțin 2 subarbori ⟹ u este punct de articulație.
- Dem (Reducere la absurd): Fie A_1 și A_2 cei 2 subarbori, $x \in A_1$, $y \in A_2$. Pp $\exists$ x..α..y și u $\notin$ x..α..y.
- z = primul nod descoperit de DFS din care se poate ajunge la x și la y. Cf. T drumurilor albe x,y ∈ Arb(z).
- Dar x,y∈Arb(u) ➔ 2 cazuri:



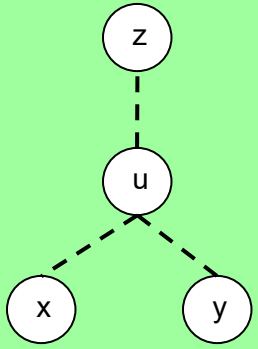
Pp $\exists$ x..α..y și u$\notin$ x..α..y.

Caz 1: d(u)<d(z):



Contradictie (1) x,y nu sunt în subarbori diferiți ai lui Arb(u).

Caz 2: d(z)<d(u):



Contradictie (1), p(u) ≠ null.

Puncte de articulație. Demonstrație teoremă (Ib)

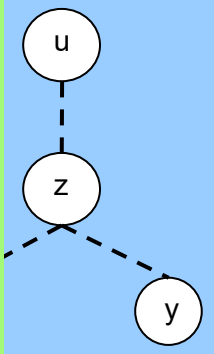
- u este punct de articulație și este descoperit în ciclul principal al DFS ⟹ p(u) = null și u domină cel puțin 2 subarbori.

- Dem (Reducere la absurd): Fie nodurile x și y a.î. u ∈ ∀ x..y. u = primul nod descoperit din cale (altfel u nu mai e descoperit în ciclul principal al DFS) => p(u) = null și x, y ∈ Arb(u).

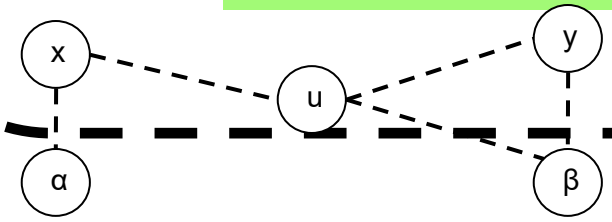
- pp că x, y
  fie z rădă
  x..z..y → u
  se contraz

```
DFS(G)
        V = noduri(G)
        Pentru fiecare nod u (u ∈ V)
                c(u) = alb; p(u) = null; // inițializare structură date
        timp = 0; // reține distanța de la rădăcina arborelui DFS până la nodul curent
        Pentru fiecare nod u (u ∈ V)
                Dacă c(u) este alb
                        Atunci explorare(u); // explorez nodul
```

celași subarbore

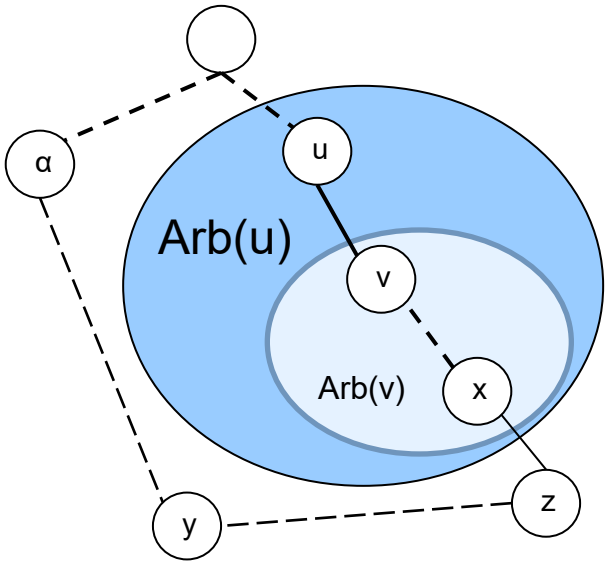


Contradicție ∃ x..z..y =>
u nu este punct de articulație



Puncte de articulație. Demonstrație teoremă (IIa)

- p(u) ≠ null și ∃ v descendent al lui u în Arb(u) a.î. ∀ x ∈ Arb(v) și ∀ (x,z) parcursă de DFS(G) are d(z) ≥ d(u) ⟹ u este punct de articulație.



Dem (Reducere la absurd): Pp. u nu e punct de articulație → ∃ w ∈ Arb(v), y ∉ Arb(u) a.î. y..w. Fie z primul nod din y..w a.î. z ∉ Arb(u) și x ultimul nod din w..y a.î. x ∈ Arb(u) → (x,z) taie frontiera Arb(u).

Dacă d(z) > d(u) → u..x, z alb la d(u) → z ∈ Arb(u) → contradicție (z ∉ Arb(u))

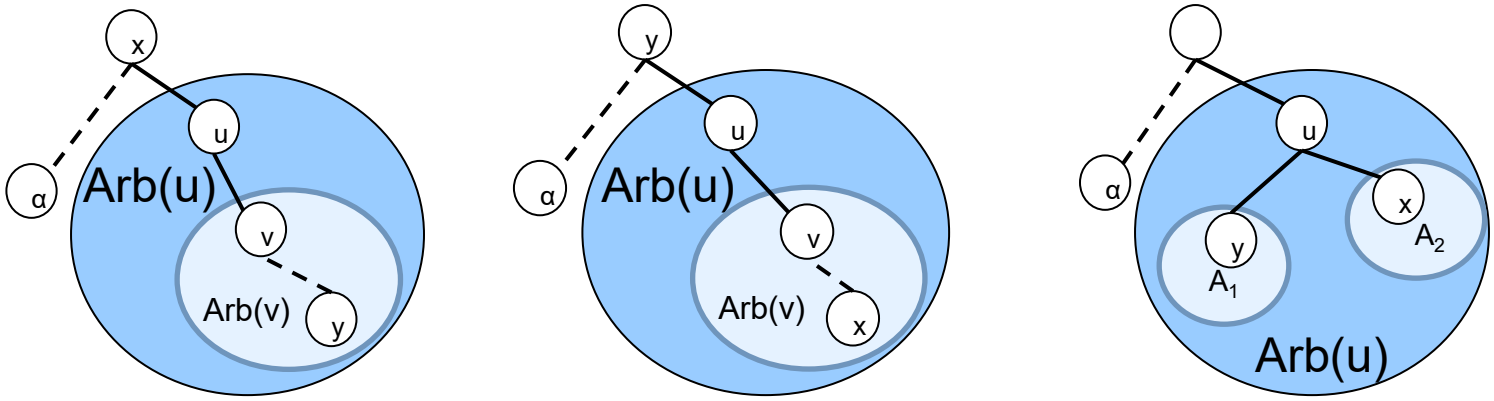
Dacă d(z) < d(u) → contradicție (ipoteza)

→ ∄ y..w → u punct de articulație

Puncte de articulație. Demonstrație teoremă (IIb)

- u este punct de articulație și nu este descoperit în ciclul principal al DFS $\Longrightarrow$ p(u) ≠ null și $\exists$ v descendent al lui u în Arb(u) a.î. $\forall$ x $\in$ Arb(v) și $\forall$ (x,z) parcursă de DFS(G) având d(z) ≥ d(u).

- Dem: Fie nodurile x și y a.î. u $\in \forall$ x..y și p(u) ≠ null. Se pot forma 3 tipuri de structuri:



- Pentru primele 2 structuri, nu trebuie sa existe muchie care sa formeze ciclu de la nici un nod din Arb(v) către vreun predecesor al lui u. Altfel $\exists$ x..y a.î. u $\notin$ x..y.

- Pentru a 3-a structura, trebuie să $\nexists$ muchie care să formeze ciclu către un predecesor al lui u de la niciun nod din cel puțin un subarbore A_1 sau A_2.

Puncte de articulație. Structuri de date.

- Structura de date de la DFS + pentru fiecare nod u ∈ V se reține:
  - Low(u) = min{d(v) | v descoperit pornind din u în cursul DFS și c(v) ≠ alb}
  - Subarb(u) = numărul subarborilor dominați de u (dacă este ≥ 2, atunci avem un punct de articulație).

Idee algoritm

- Se aplică DFS și se salvează pentru fiecare nod până unde merge înapoi (low): low[u] = min {d(u), d(v) pentru toate muchiile înapoi (u,v), low(w) pentru toți fiii w ai lui u}.

- Pentru eficiență, trebuie ca fiii să se parcurgă înaintea părinților → ordinea inversă a d(u).



Algoritm Tarjan (I)

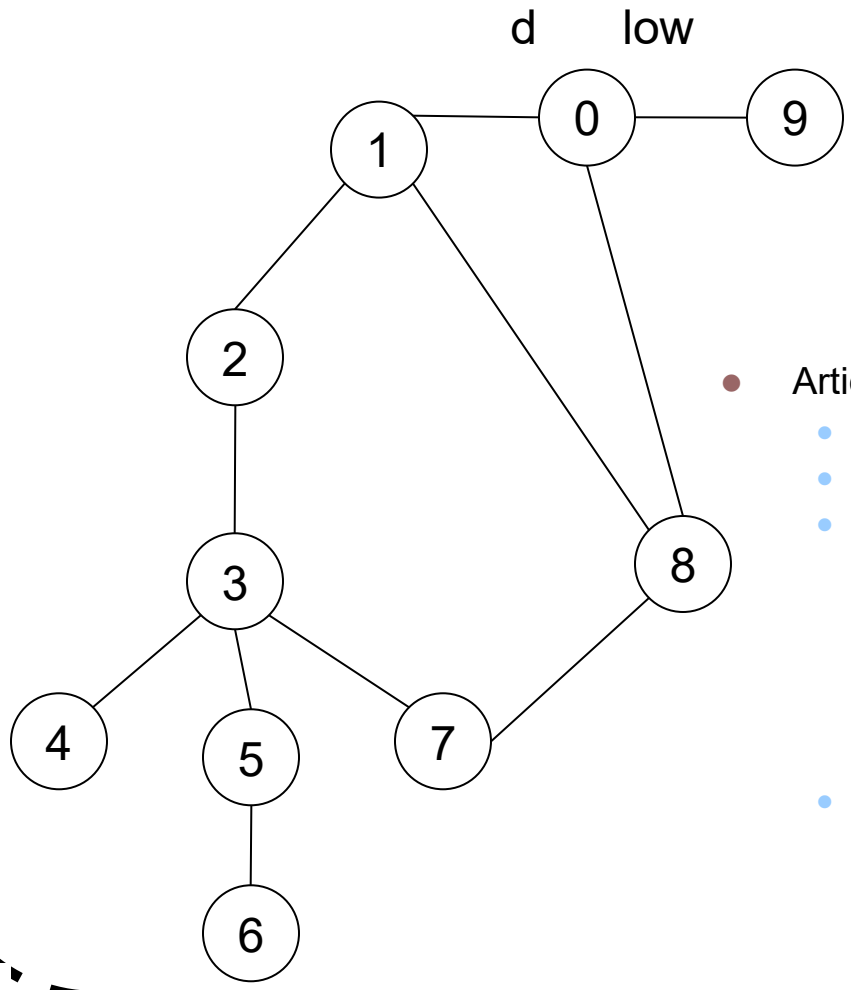
- Articulații (G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - subarb(u) = 0; // reține numărul de subarbori dominați de u
    - art(u) = 0; // reține punctele de articulație
  - **Pentru fiecare** (u∈V)
    - **Dacă** c(u) e alb
      - Exploreaza(u);
      - **Dacă** (subarb(u) > 1) // cazul în care u este rădăcina în arborele
        - art(u) = 1 // DFS și are mai mulți subarbori → cazul // 1 al teoremei

Algoritm Tarjan (II)

low[u] = min {d(u), d(v) pentru toate muchiile înapoi (u,v), low(w) pentru toți fiii w ai lui u}

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori // dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) ≠ null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

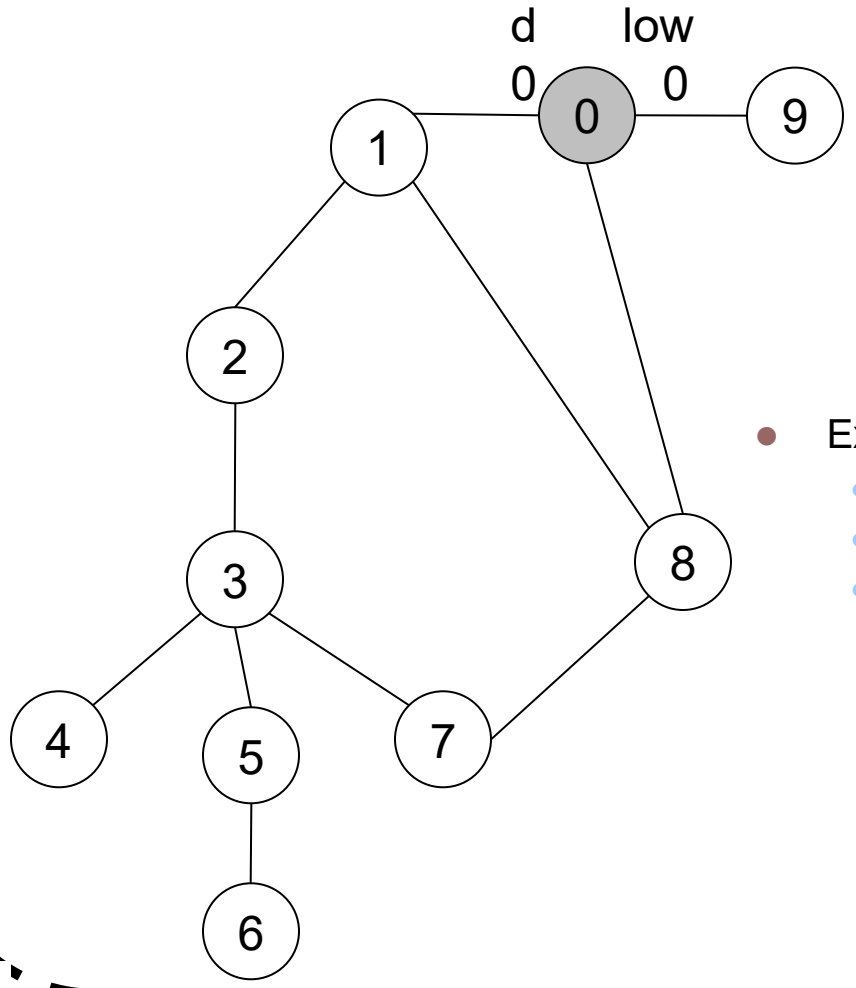
Exemplu rulare (1)



Timp = 0
C(i) = alb
D(i) = 0
Low(i) = 0
P(i) = null
Subarb(i) =0
Art(i) = 0
Exploreaza (0)

- Articulații (G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - subarb(u) = 0; // reține numărul de subarbori dominați de u
    - art(u) = 0; // reține punctele de articulație
  - **Pentru fiecare** (u∈V)
    - **Dacă** c(u) e alb
      - Exploreaza(u);
      - **Dacă** (subarb(u) > 1) // cazul în care u este rădăcina în
        - art(u) = 1 // arborele DFS și are mai mulți // subarbori → cazul 1 al teoremei

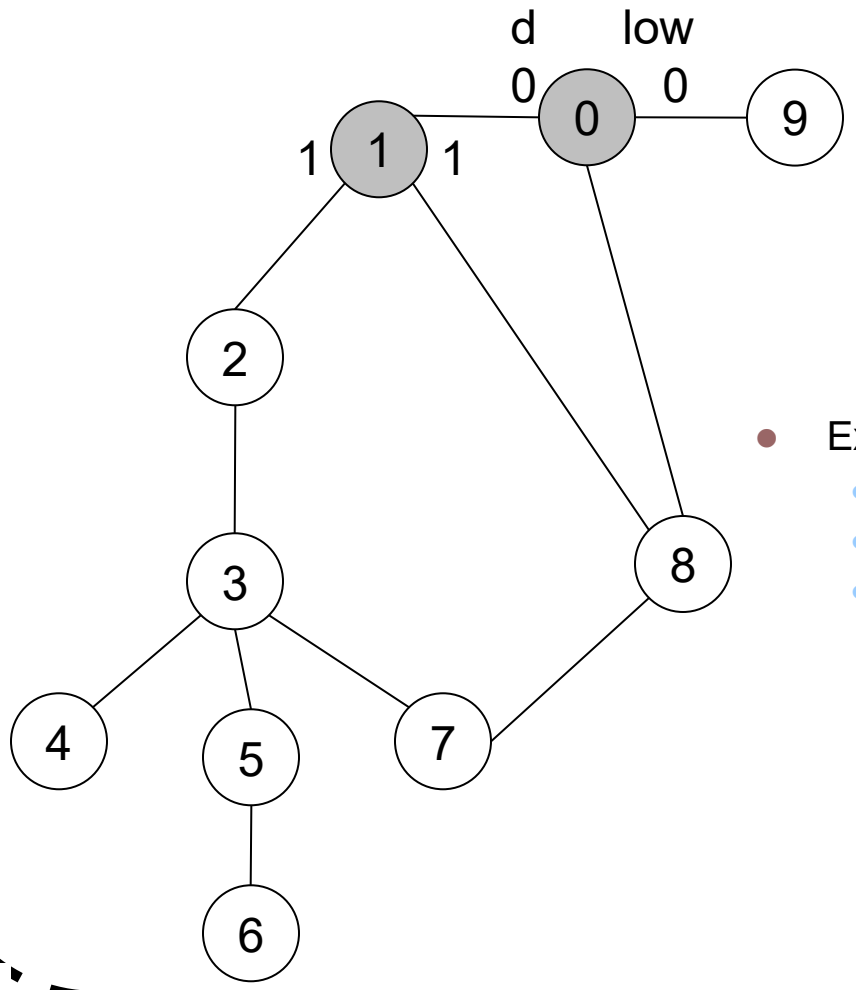
Exemplu rulare (2)



Low(0) = d(0) = 0
Timp = 1
C(0) = gri
P(1) = 0
Subarb(0) = 1
Exploreaza (1)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

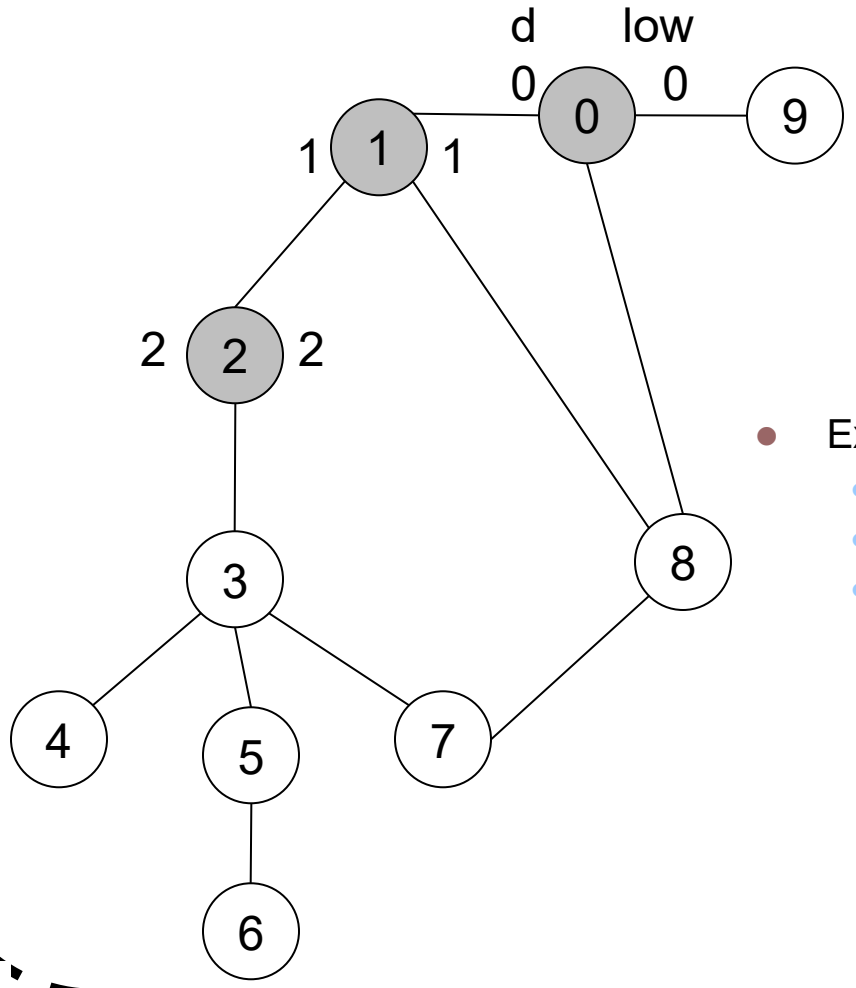
Exemplu rulare (3)



Low(1) = d(1) = 1
Timp = 2
C(1) = gri
P(2) = 1
Subarb(1) = 1
Exploreaza (2)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

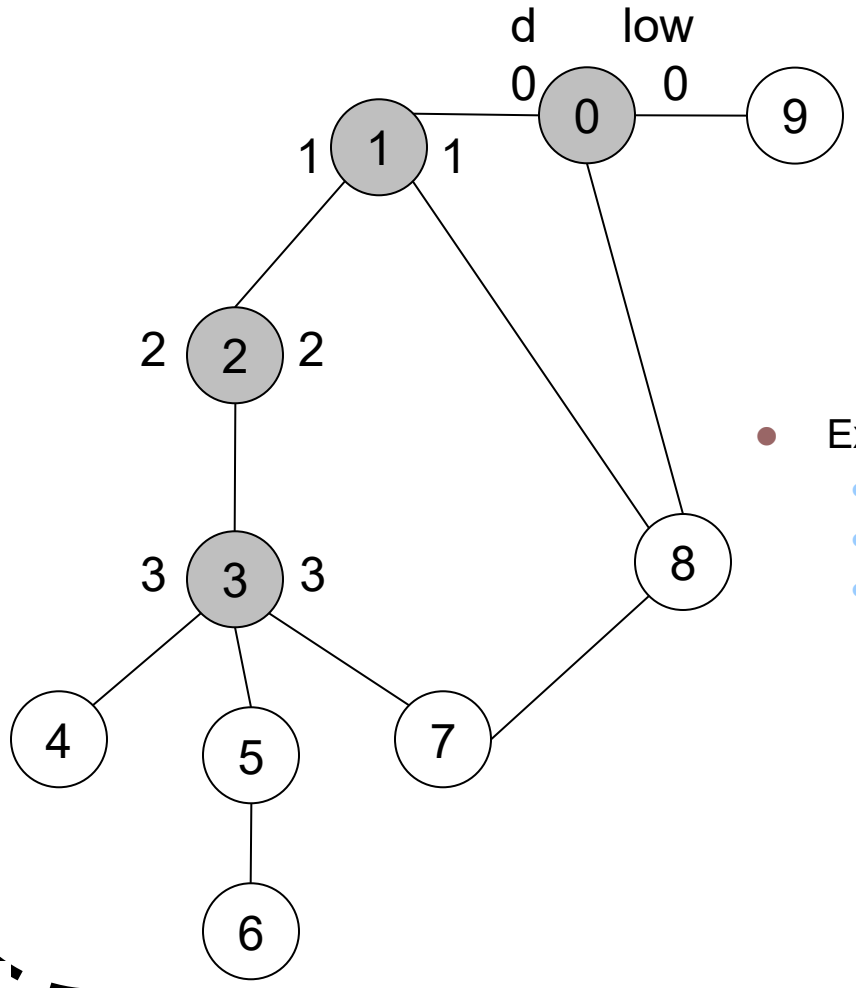
Exemplu rulare (4)



Low(2) = d(2) = 2
Timp = 3
C(2) = gri
P(3) = 2
Subarb(2) = 1
Exploreaza (3)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

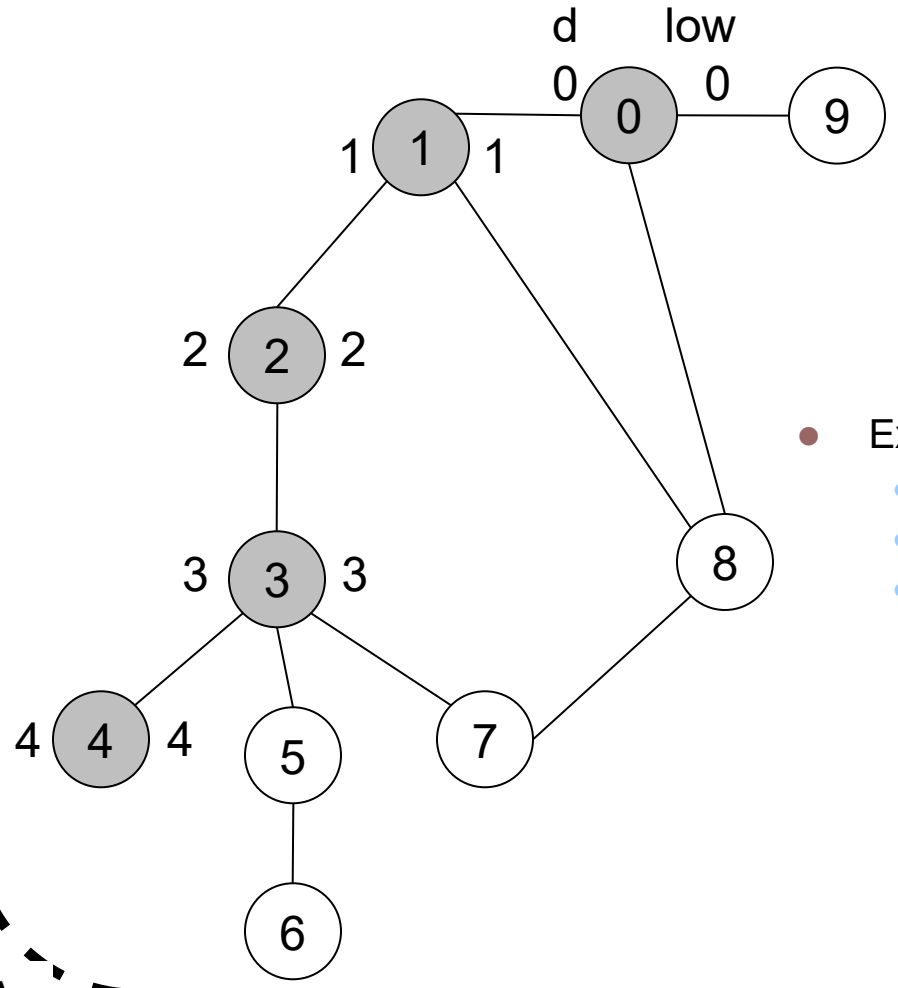
Exemplu rulare (5)



Low(3) = d(3) = 3
Timp = 4
C(3) = gri
P(4) = 3
Subarb(3) = 1
Exploreaza (4)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

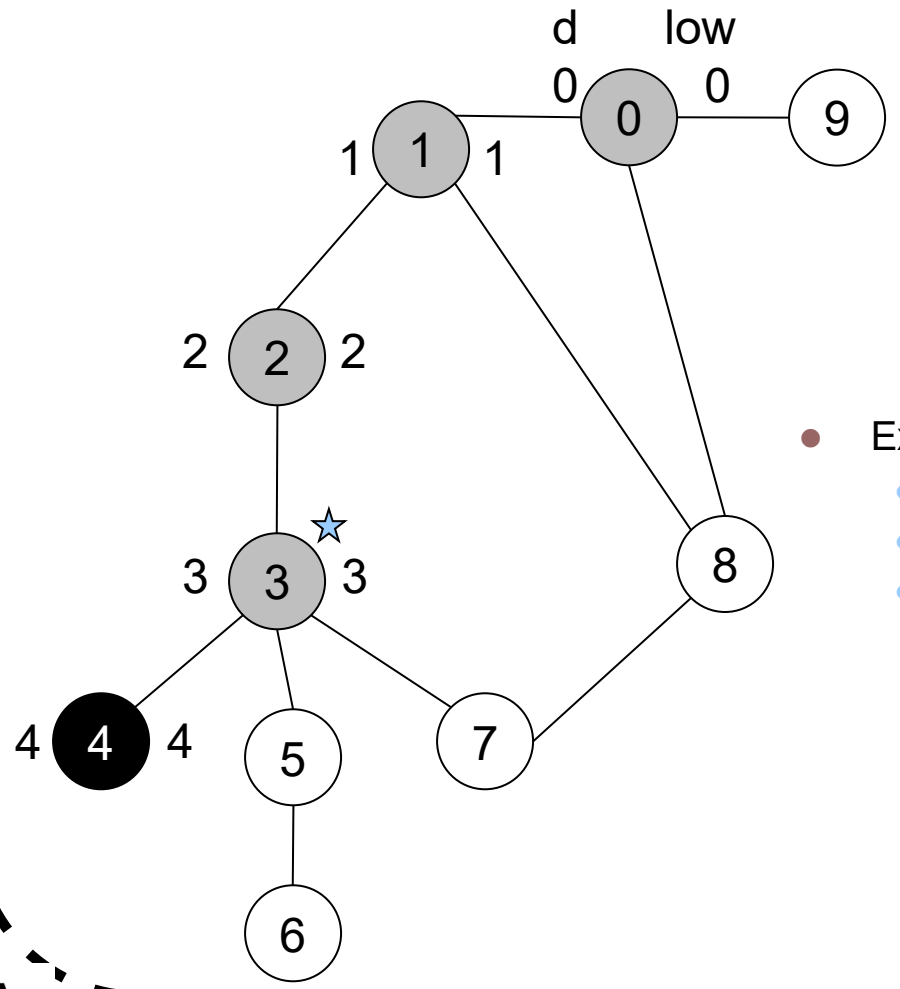
Exemplu rulare (6)



Low(4) = d(4)=4
Timp =5
C(4) =gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

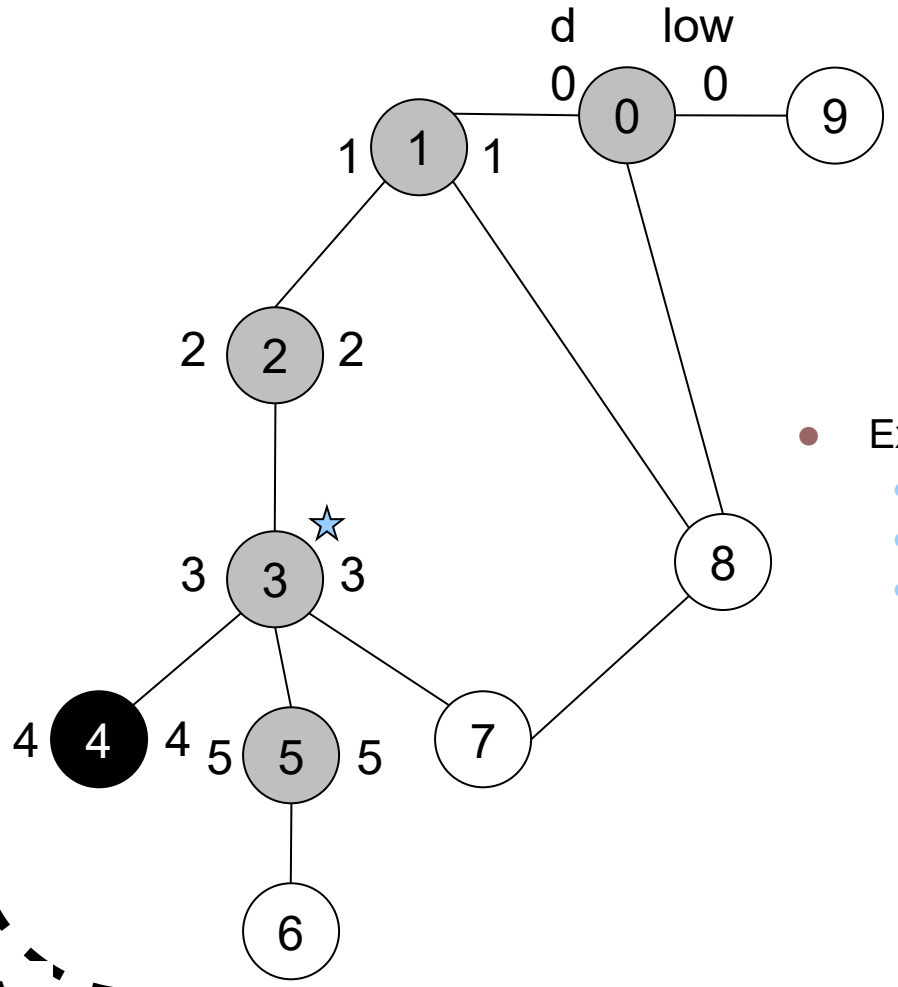
Exemplu rulare (7)



Low(4) = d(4) = 4
Timp = 5
C(4) = gri
revenire
Low (3) = min {low(3), low(4)} = 3
Low(4) > d(3) → art(3) = 1
P(5) = 3
Subarb(3) = 2
Exploreaza (5)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

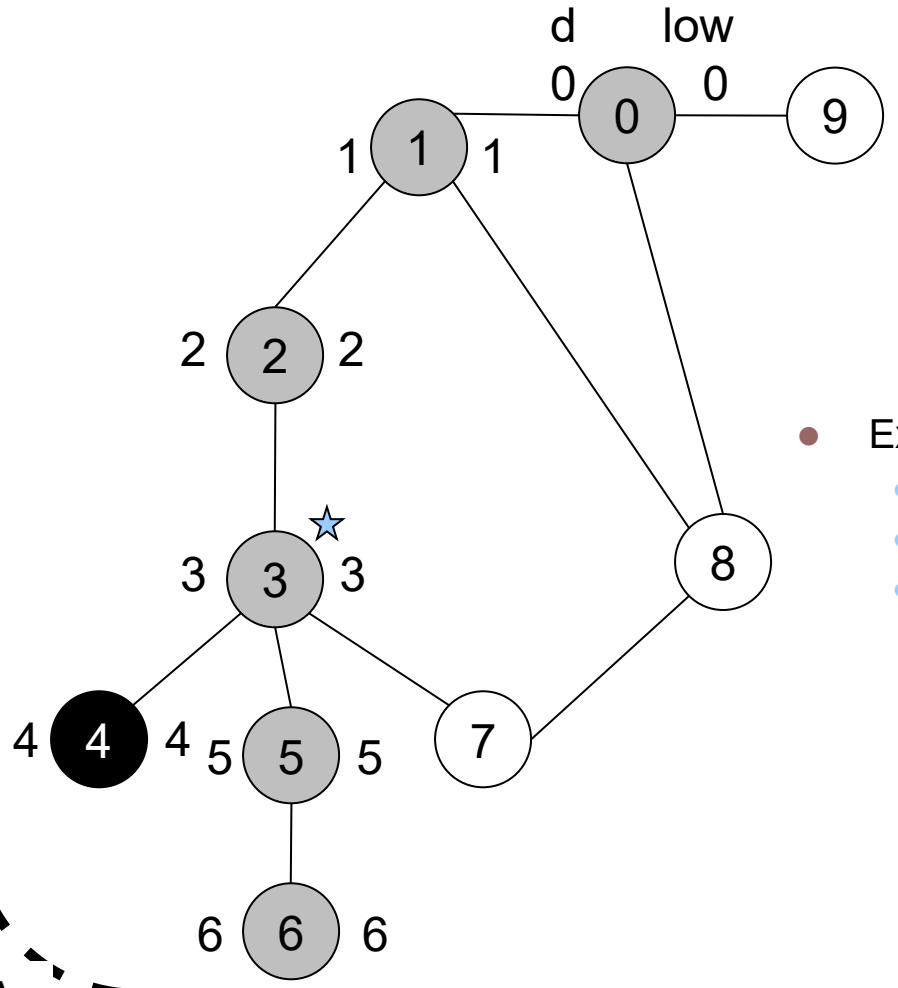
Exemplu rulare (8)



Low(5) = d(5) = 5
Timp = 6
C(5) = gri
P(6) = 5
Subarb(5) = 1
Exploreaza (6)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

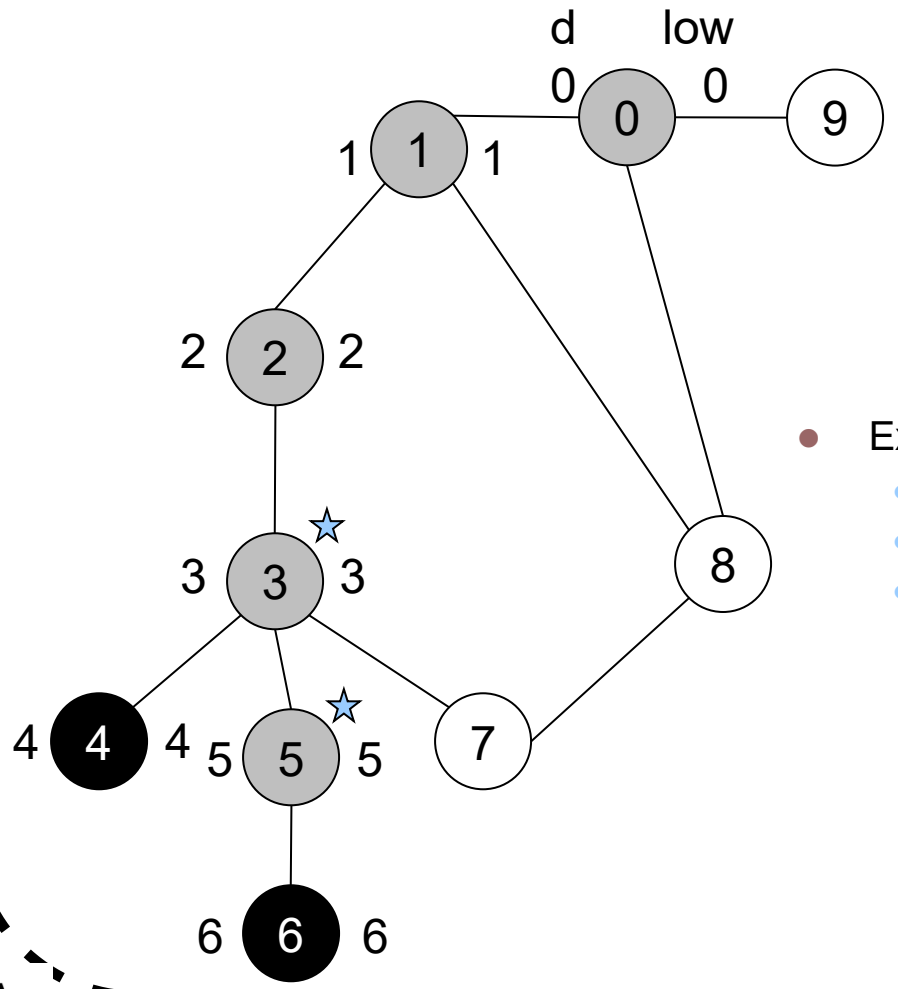
Exemplu rulare (9)



Low(6) = d(6) = 6
Timp = 7
C(6) = gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

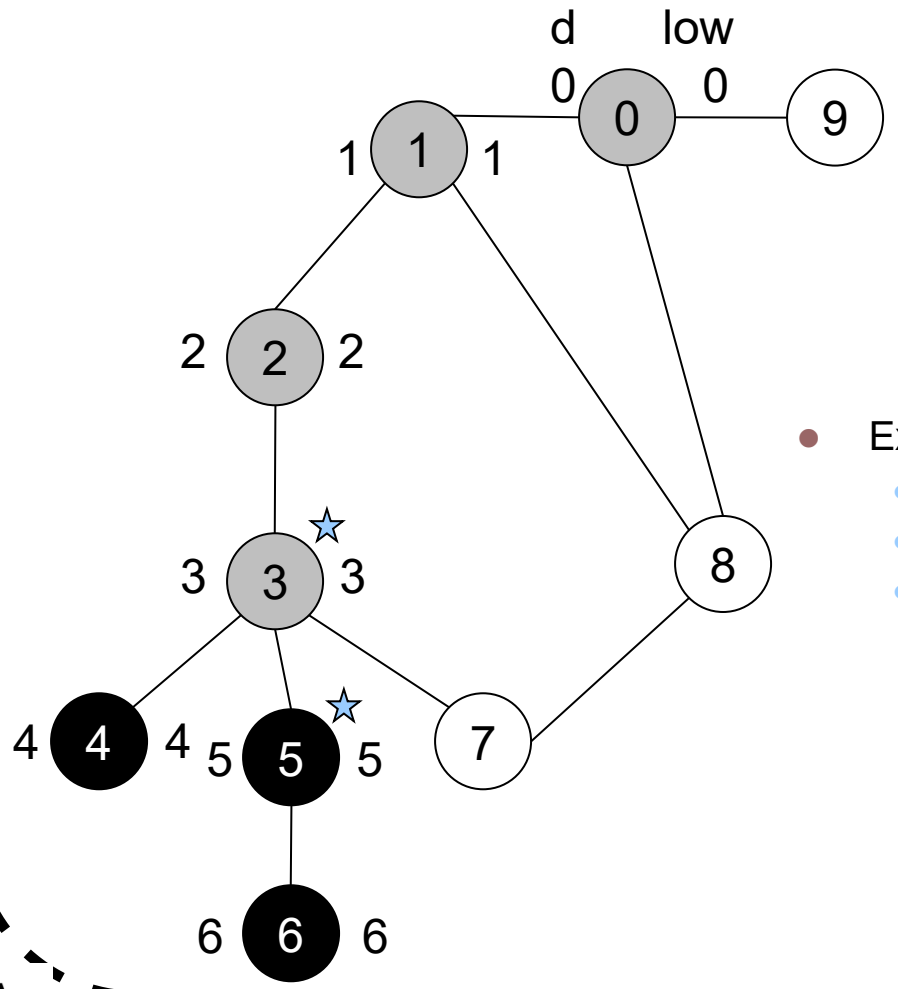
Exemplu rulare (10)



Low(6) = d(6) = 6
Timp = 7
C(6) = gri
revenire
Low (5) = min {low(5), low(6)} = 5
Low(6) > d(5) → art(5) = 1
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

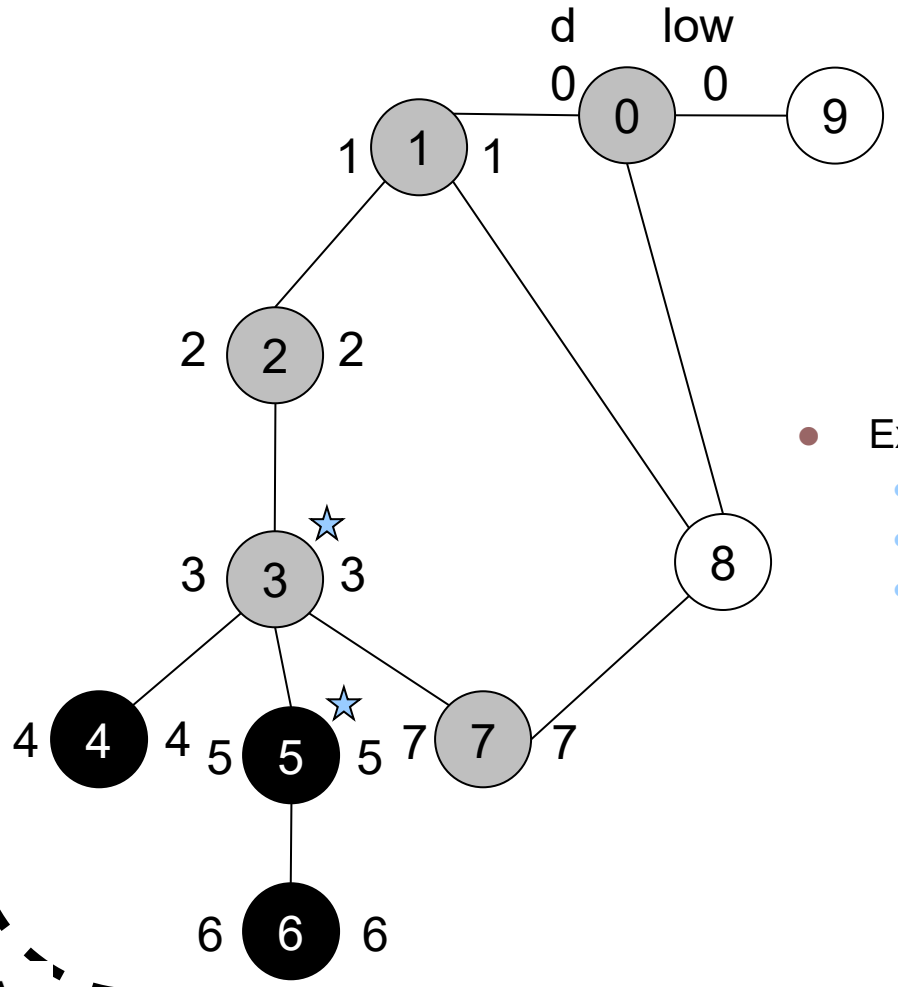
Exemplu rulare (11)



Low(5) = d(5) = 5
Timp = 7
C(5) = gri
revenire
Low (3) = min {low(3), low(5)} = 3
Low(5) > d(3) → art(3) = 1
P(7) = 3
Subarb(3) = 3
Exploreaza (7)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

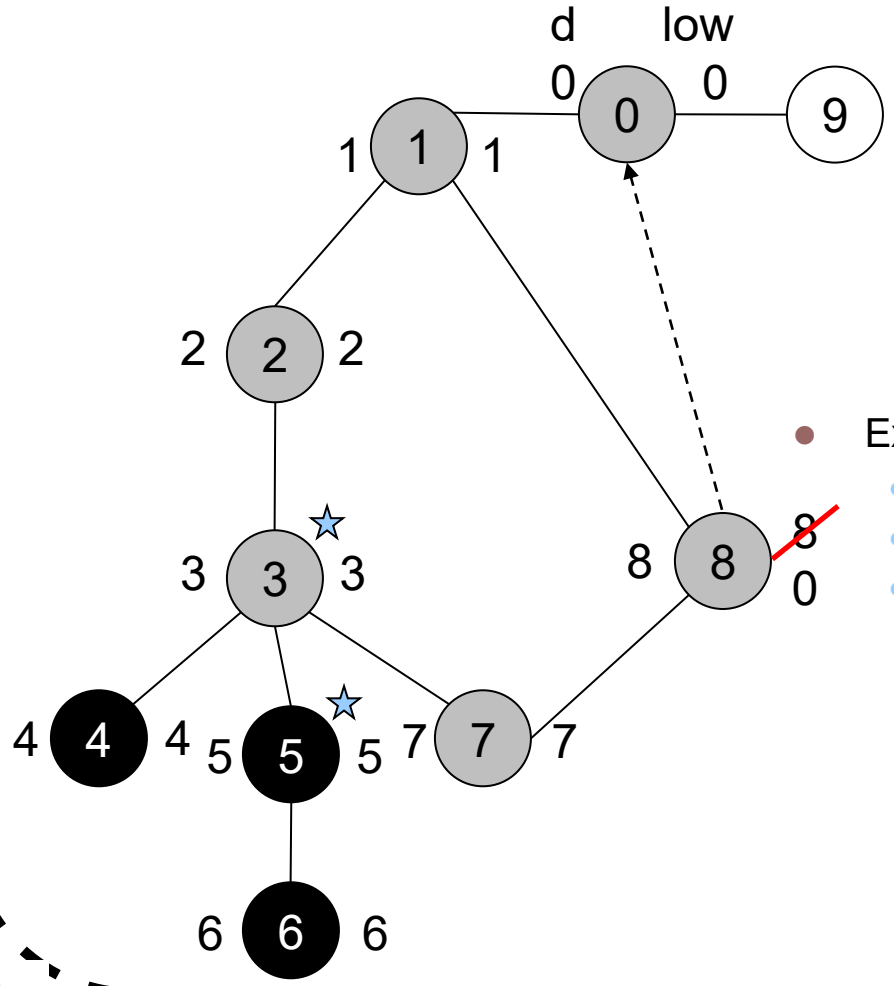
Exemplu rulare (12)



Low(7) = d(7) = 7
Timp = 8
C(7) = gri
P(8) = 7
Subarb(7) = 1
Exploreaza (8)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

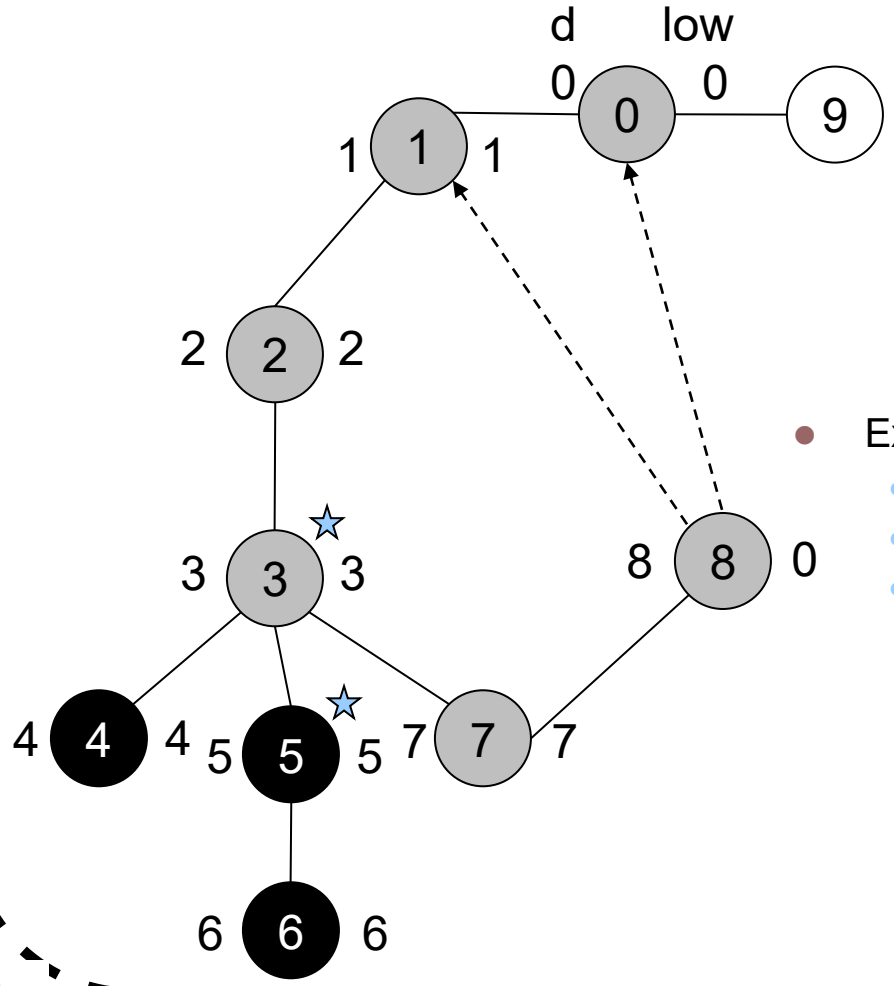
Exemplu rulare (13)



Low(8) = d(8) = 8
Timp = 9
C(8) = gri
Low(8) = min{d(0), low(8)} = 0

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

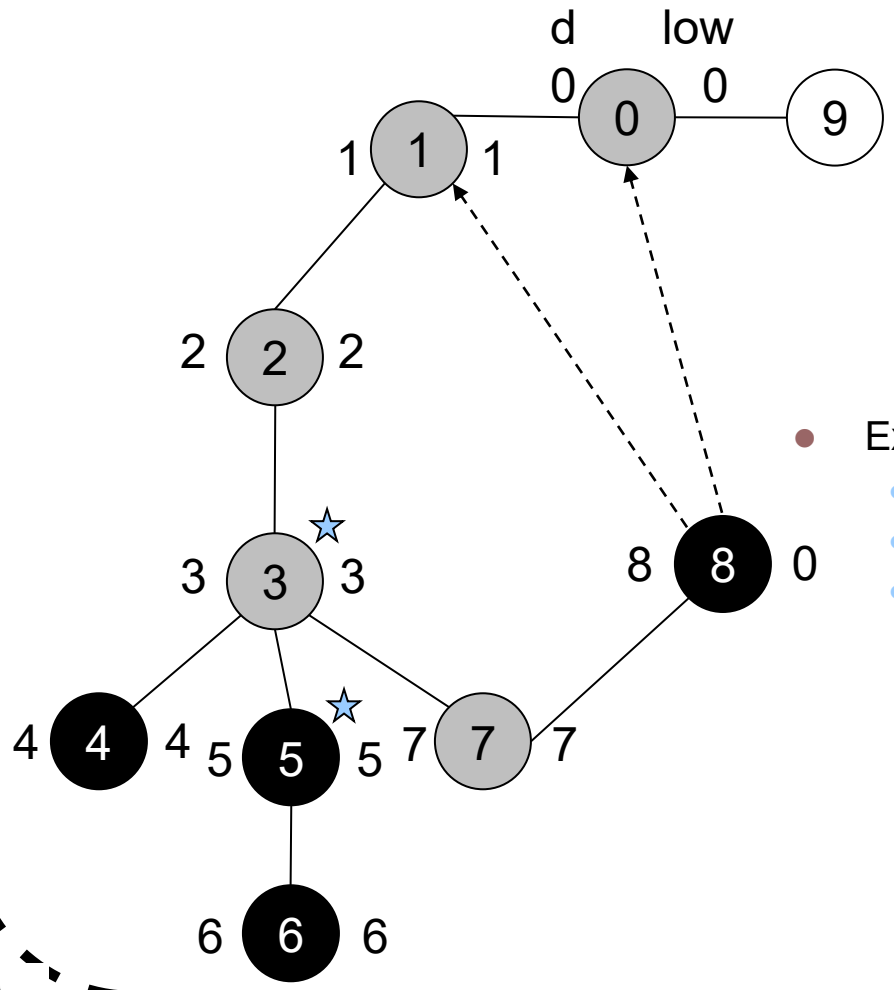
Exemplu rulare (14)



d(8) = 8
Low(8) = 0
Timp = 9
C(8) = gri
Low(8) = min{d(1), low(8)} = 0

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

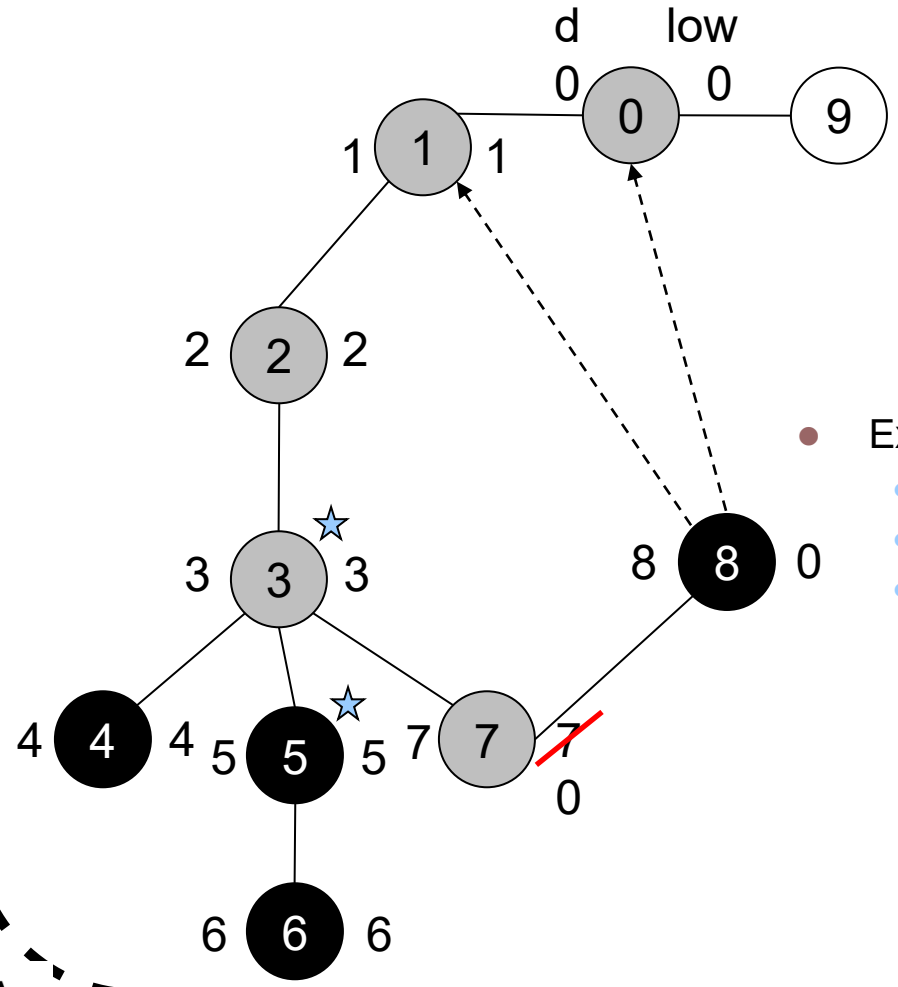
Exemplu rulare (15)



d(8) = 8
Low(8) = 0
Timp = 9
C(8) = gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

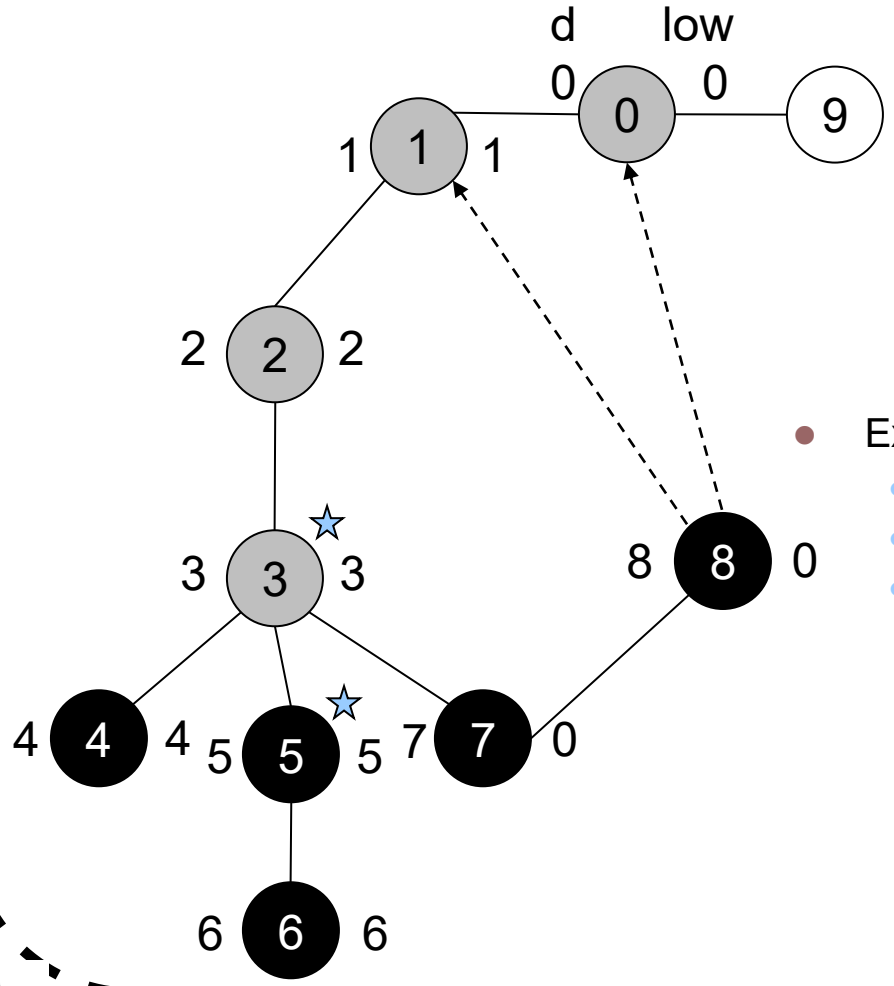
Exemplu rulare (16)



low(7) = min(low(7), low(8)) = 0
low(8) < d(7) → nu se modifică art(7)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

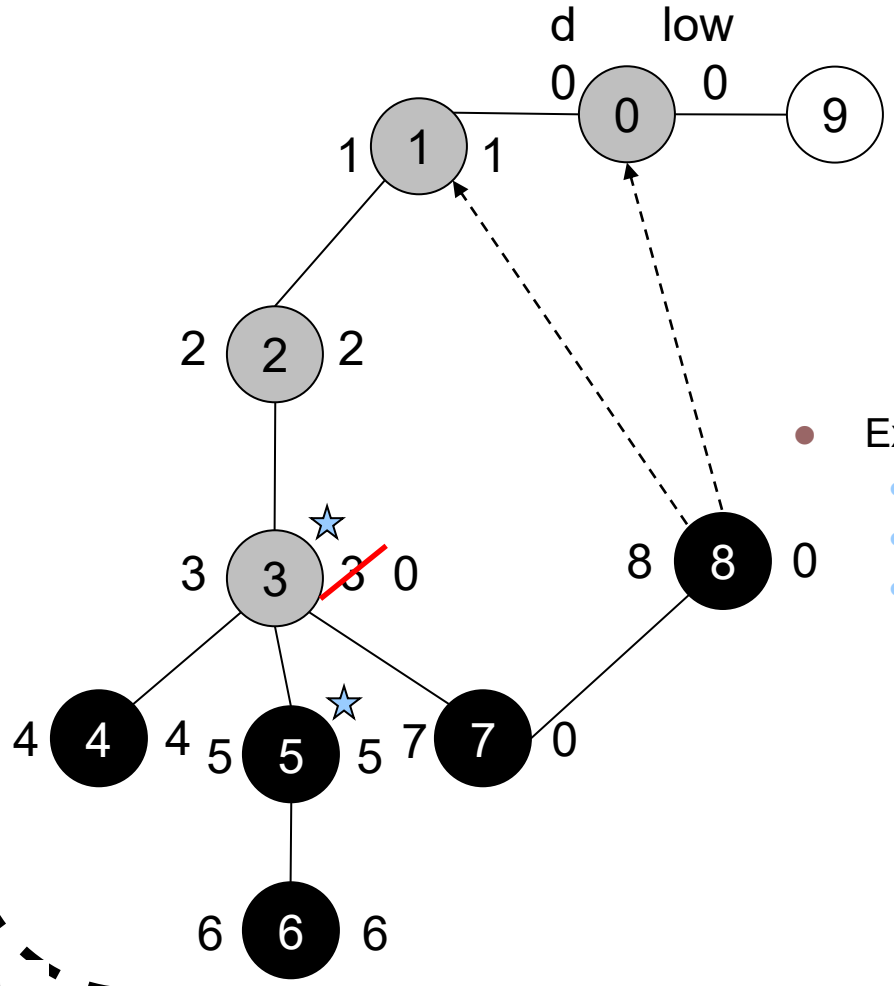
Exemplu rulare (17)



low(7) = min(low(7), low(8)) = 0
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

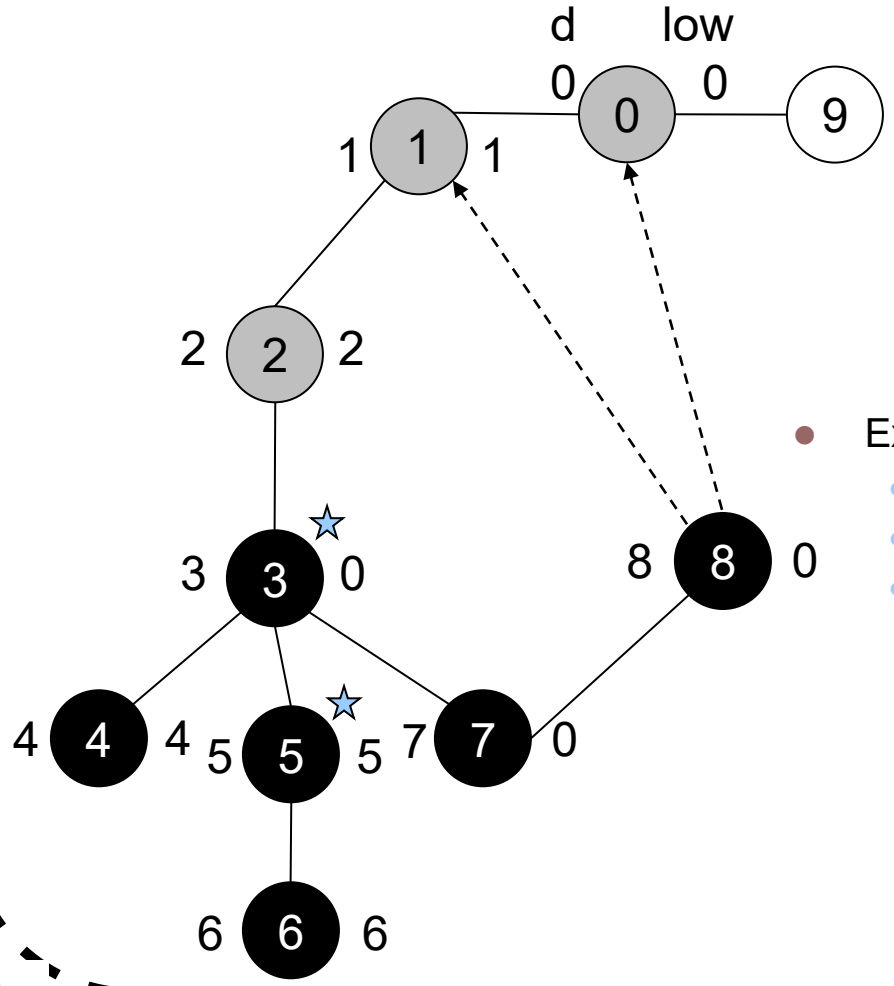
Exemplu rulare (18)



low(3) = min(low(3), low(7)) = 0
low(7) < d(3) → nu se modifică art(3)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

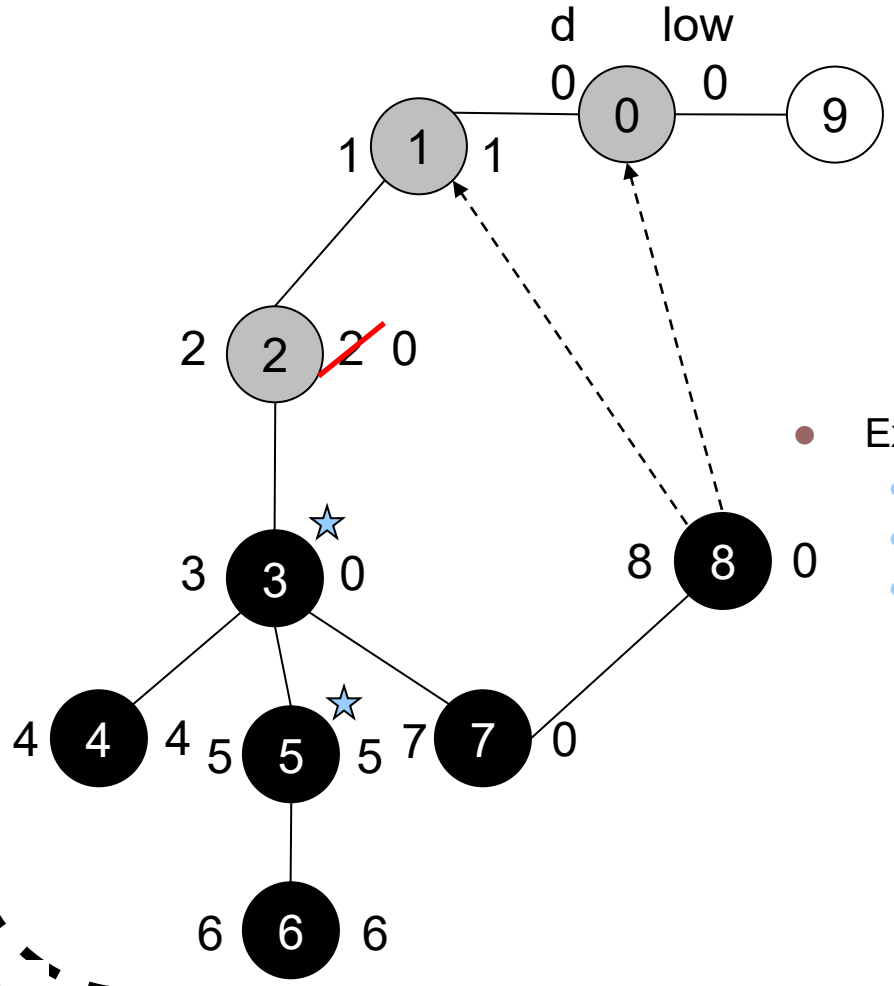
Exemplu rulare (19)



low(3) = min(low(3), low(7)) = 0
low(7) < d(3) → nu se modifică
art(3)
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

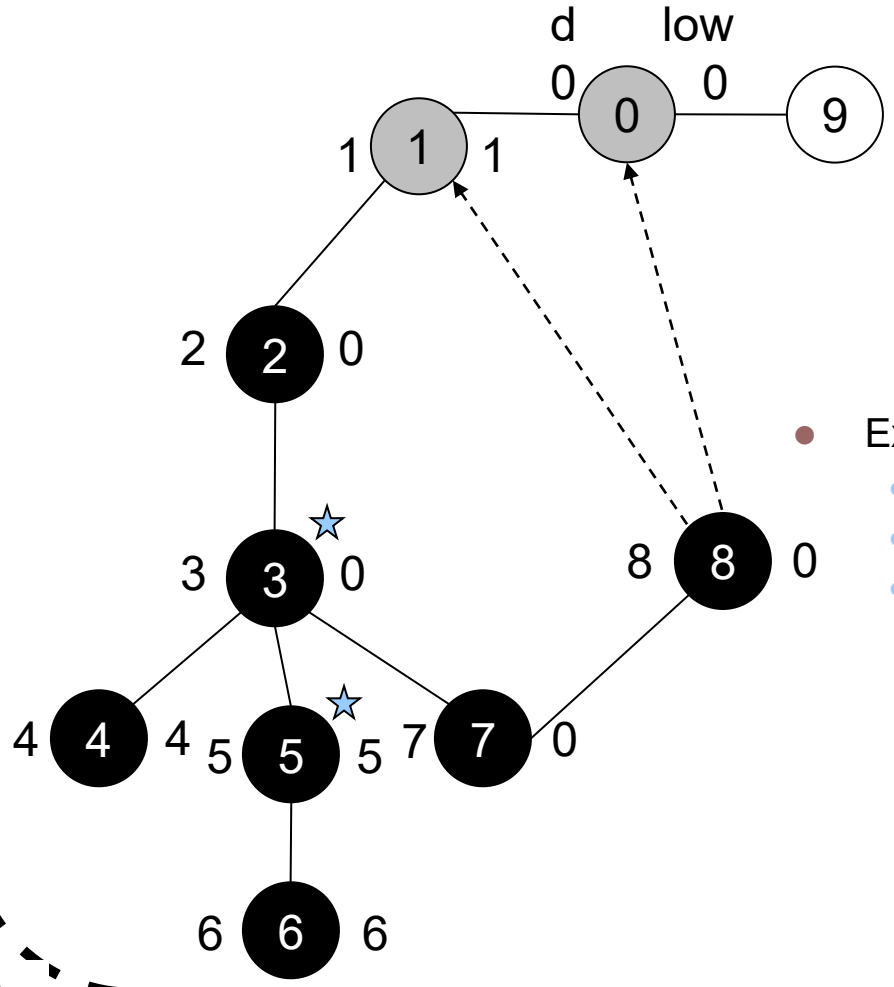
Exemplu rulare (20)



low(2) = min(low(2), low(3)) = 0
low(3) < d(2) → nu se modifică art(2)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

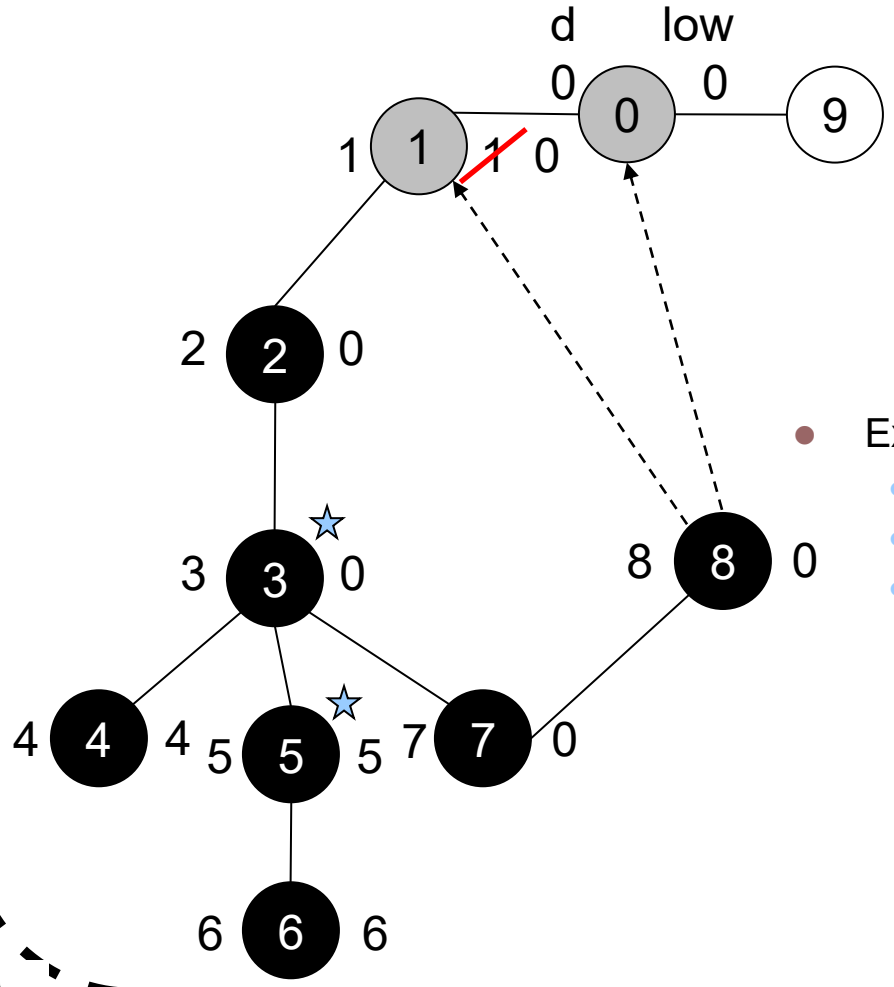
Exemplu rulare (21)



low(2) = min(low(2), low(3)) = 0
low(3) < d(2) → nu se modifică
art(2)
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

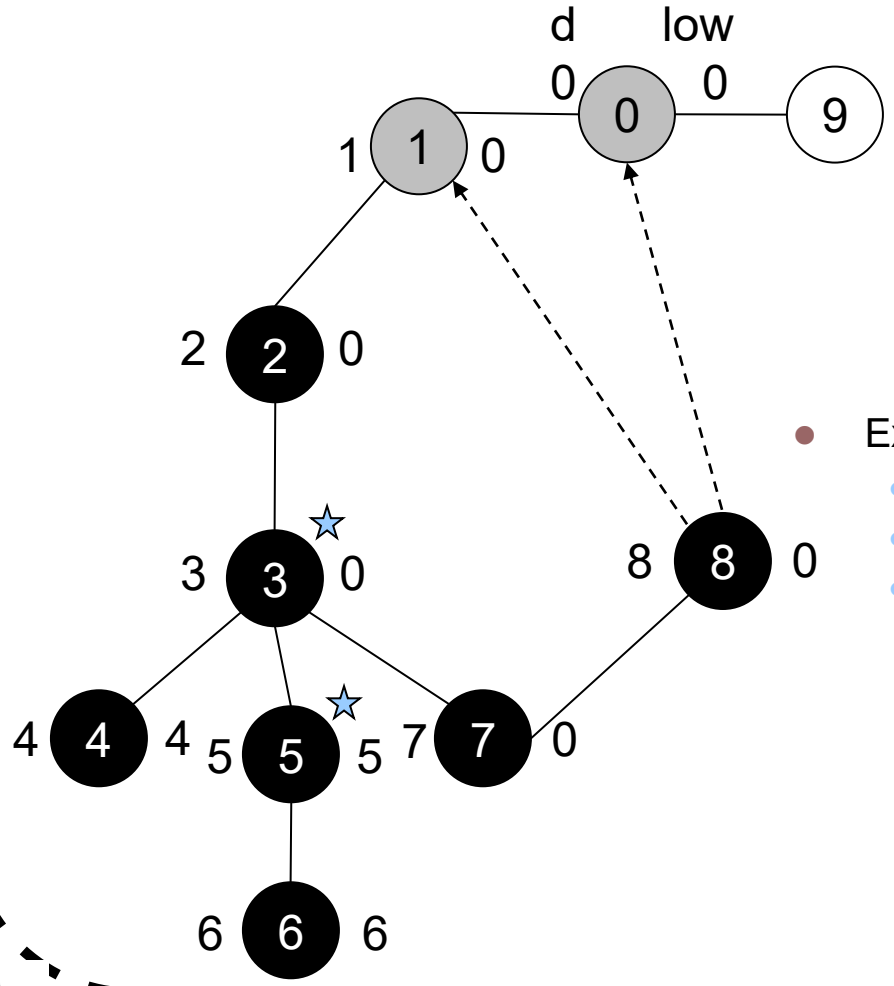
Exemplu rulare (22)



low(1) = min(low(1), low(2)) = 0
low(2) < d(1) → nu se modifică art(1)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

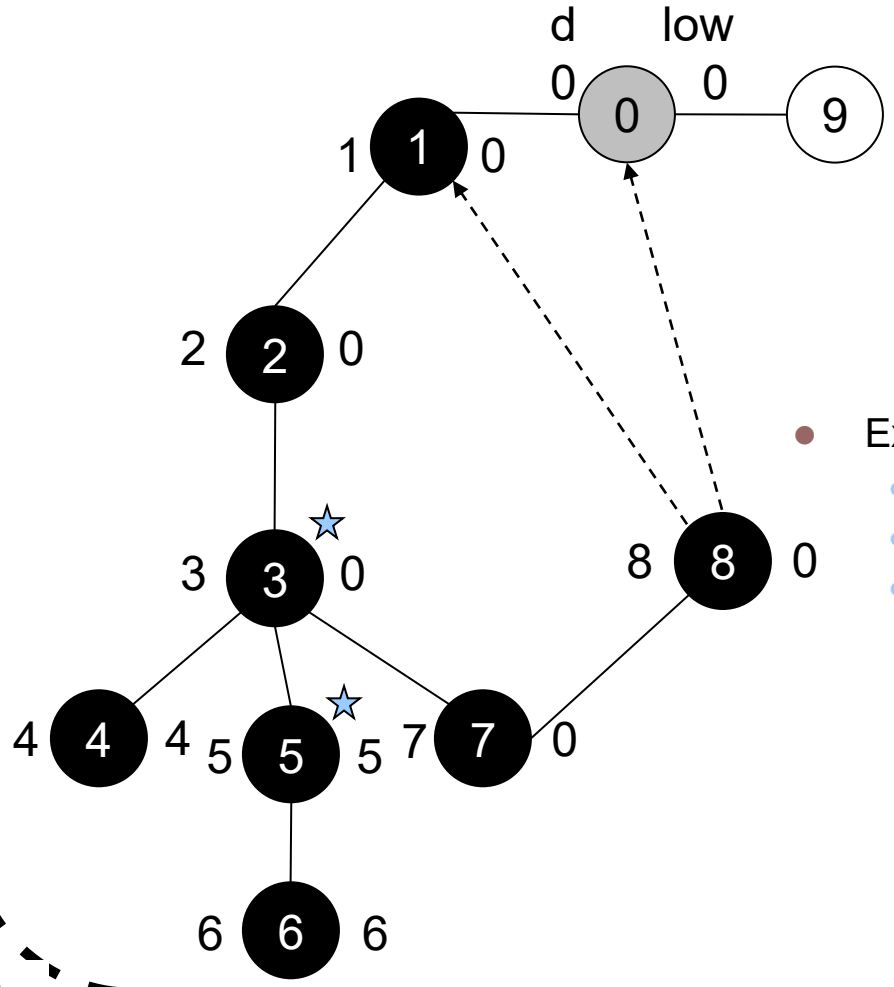
Exemplu rulare (23)



low(1) = 0
low(1) = min(low(1), d(8)) = 0

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

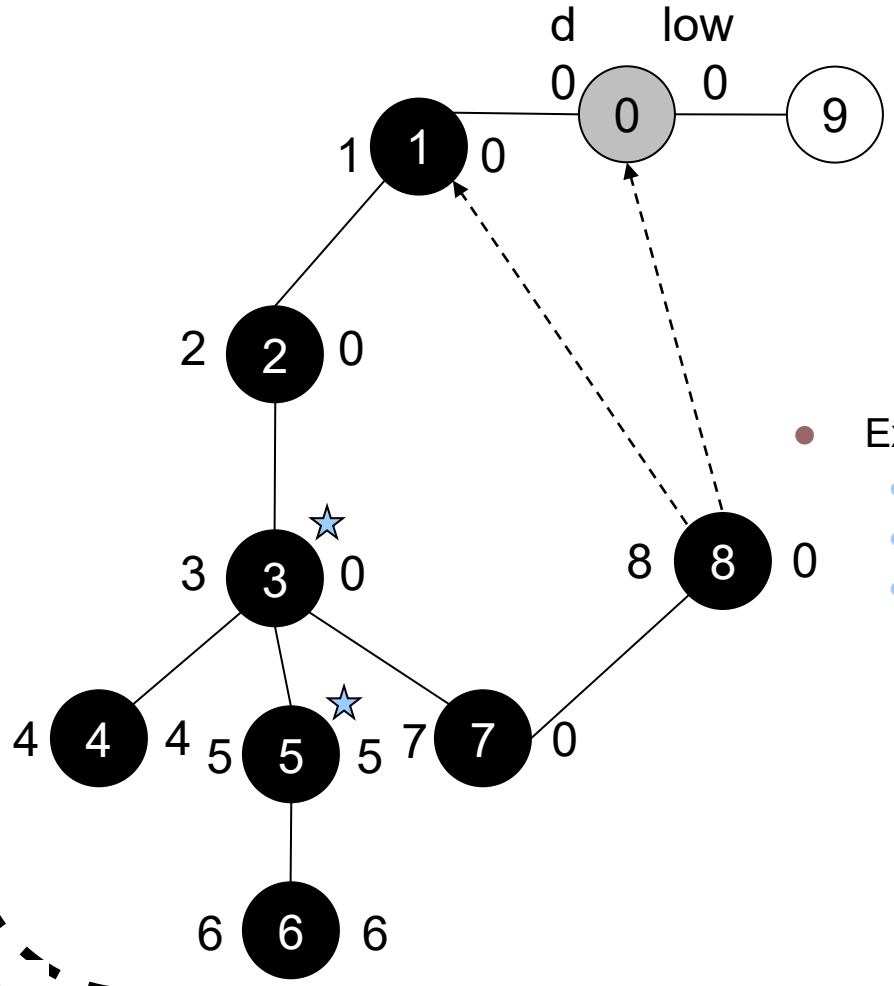
Exemplu rulare (24)



low(1) = 0
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

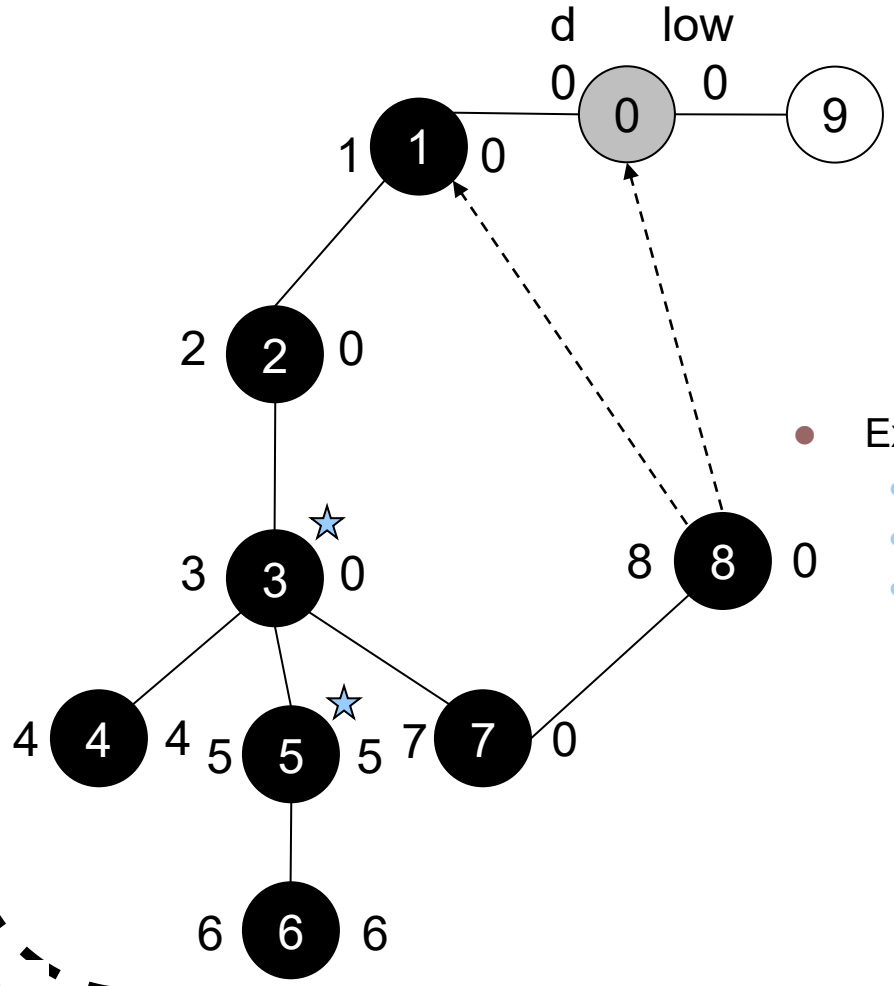
Exemplu rulare (25)



low(0) = min{low(1), low(0)} = 0
P(0) = null → continuă cu următorul copil (8)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

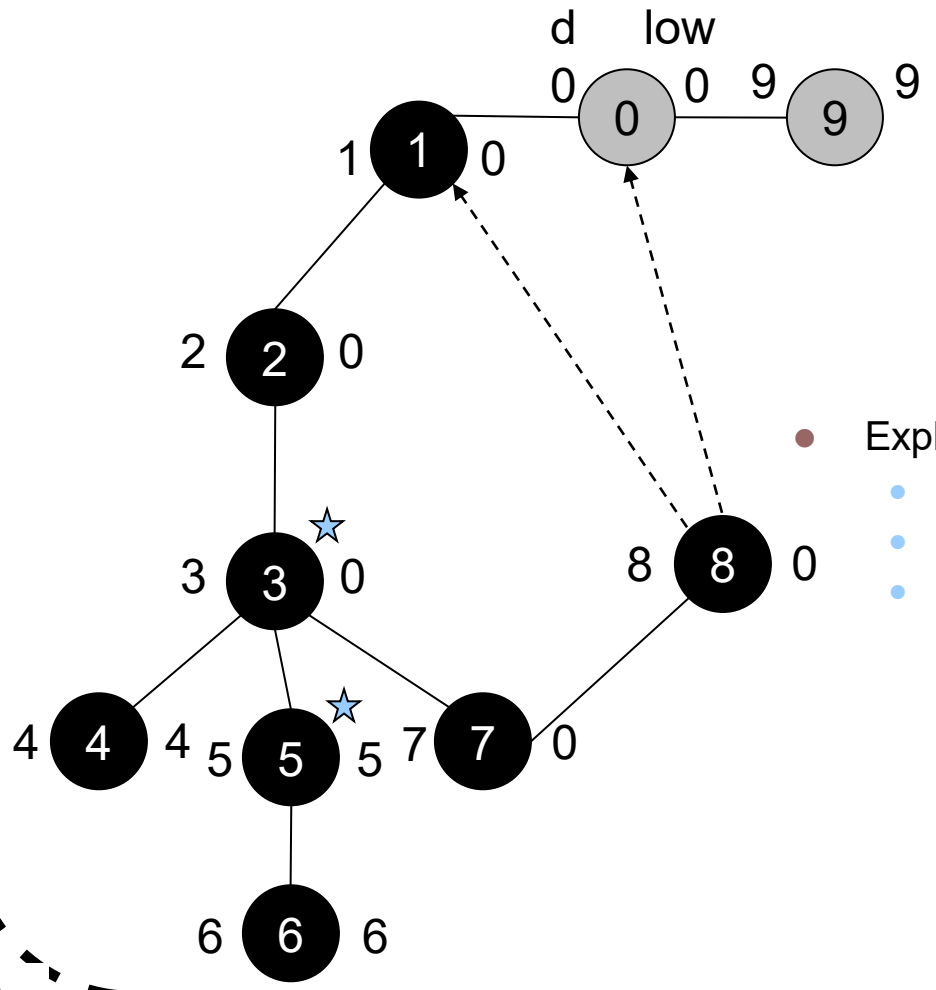
Exemplu rulare (26)



low(0) = min(low(0), d(8)) = 0
P(9) = 0
Subarb(0) = 2
Exploreaza (9)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

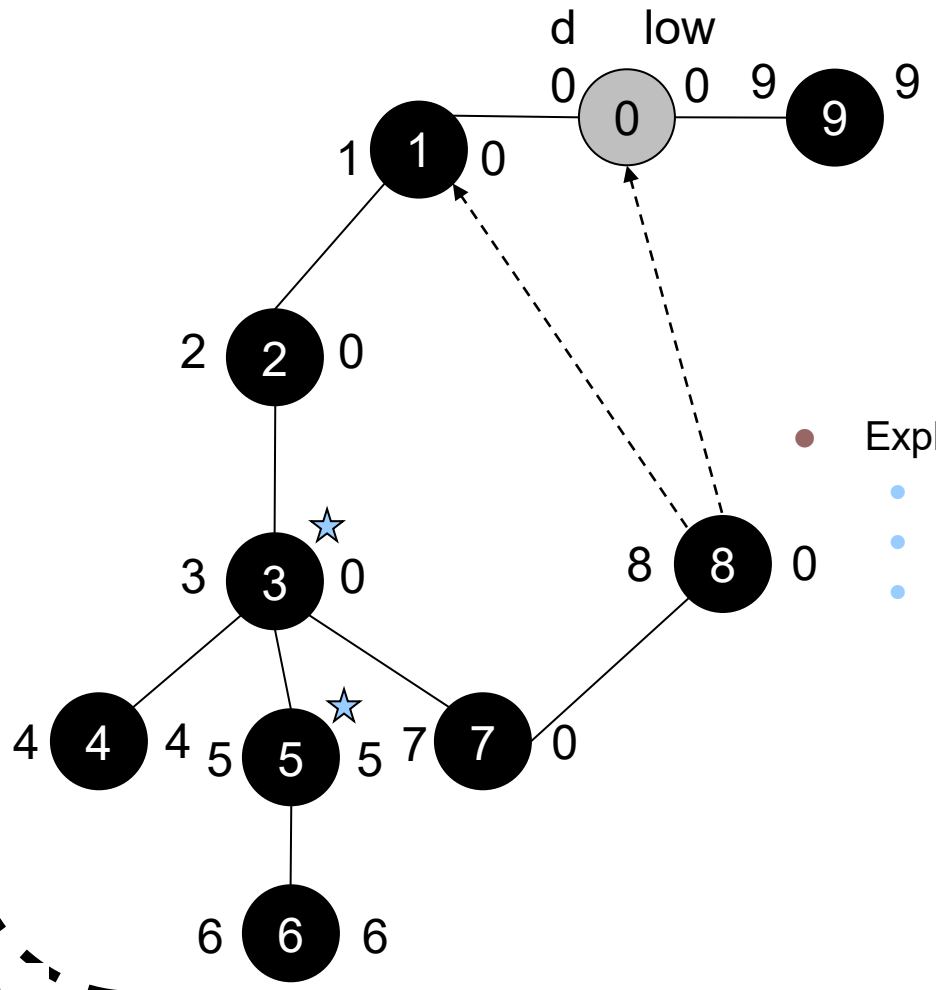
Exemplu rulare (27)



Low(9) = d(9) = 9
Timp = 10
C(9) = gri
revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

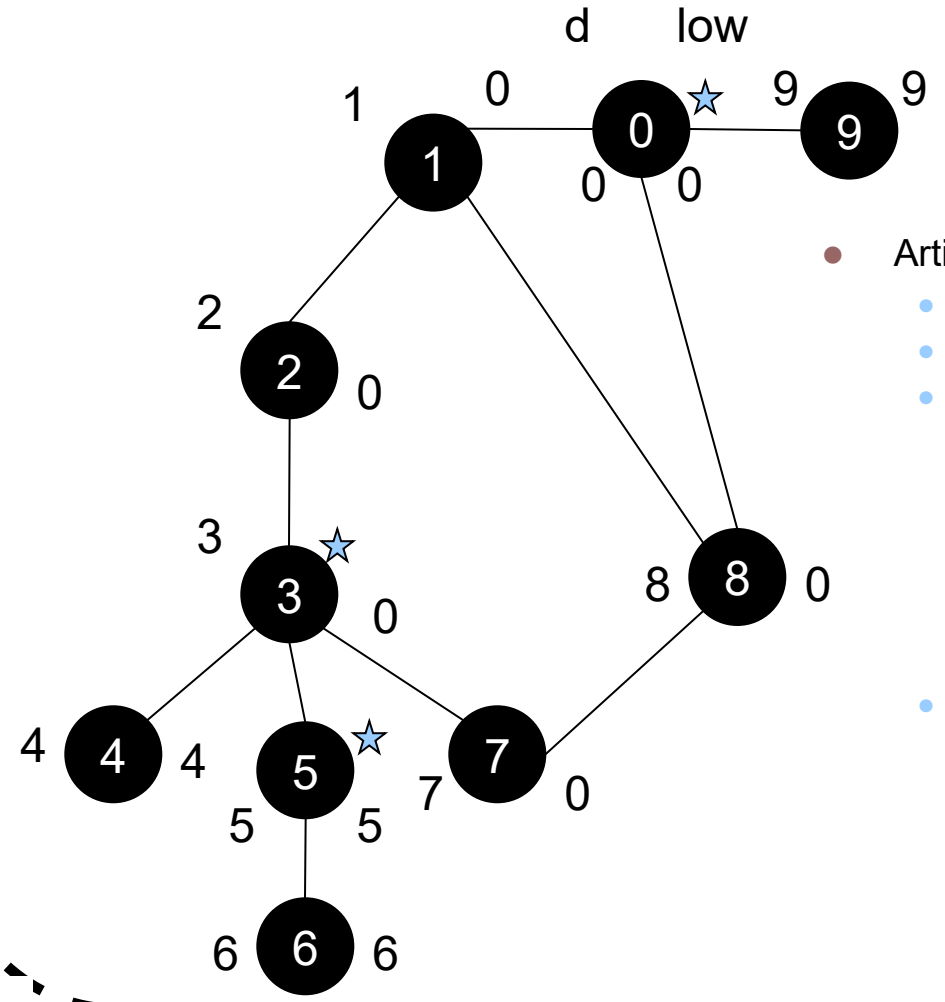
Exemplu rulare (28)



low(0) = min(low(0), low(9)) = 0
P(0) = null → revenire

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr // subarbori dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) != null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

Exemplu rulare (29)



Subarb(0) = 2 > 1 → art(0) = 1

- Articulații (G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - subarb(u) = 0; // reține numărul de subarbori // dominați de u
    - art(u) = 0; // reține punctele de articulație
  - **Pentru fiecare** (u∈V)
    - **Dacă** c(u) e alb
      - Exploreaza(u);
      - **Dacă** (subarb(u) > 1) // cazul în care u este // rădăcina în arborele art(u) = 1 // DFS și are mai mulți subarbori // → cazul 1 al teoremei

Algoritmul lui Tarjan adaptat pentru determinarea CTC

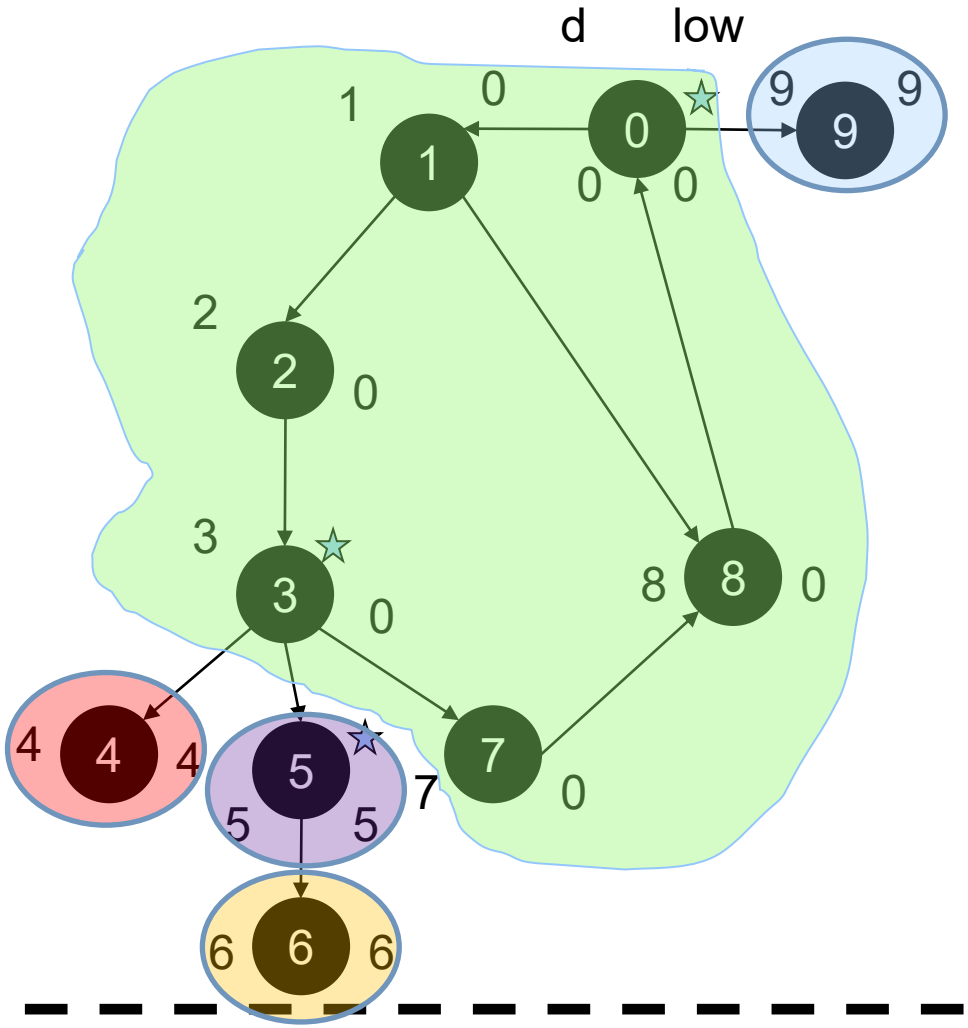
- index = 0 // nivelul pe care este nodul în arborele DFS
- S = empty // se folosește o stivă care se inițializează cu Ø
- **Pentru fiecare** v din V
  - **Dacă** (v.index e nedefinit) **atunci** // se pornește DFS din fiecare nod pe care
    - Tarjan(v) // nu l-am vizitat încă

- Tarjan(v)
  - v.index = index // se setează nivelul nodului v
  - v.lowlink = index // reține strămoșul nodului v
  - index = index + 1 // incrementez nivelul
  - S.push(v) // introduc v în stivă
  - **Pentru fiecare** (v, v') din E // se prelucrează succesorii lui v
    - **Dacă** (v'.index e nedefinit **sau** v' e în S) **atunci** // CTC deja identificate sunt ignorate
      - **Dacă** (v'.index e nedefinit) **atunci** Tarjan(v') // dacă nu a fost vizitat v' intru în recursivitate
      - v.lowlink = min(v.lowlink, v'.lowlink) //actualizez strămoșul
  - **Dacă** (v.lowlink == v.index) **atunci** // printez CTC începând de la coadă spre rădăcină
    - print "CTC:"
    - **Repetă**
      - v' = S.pop // extrag nodul din stiva și îl printez
      - print v'
    - **Până când** (v' == v) // până când extrag rădăcina

low[u] = min {d(u), d(v) pentru toate muchiile înapoi (u,v), low(w) pentru toți fiii w ai lui u}

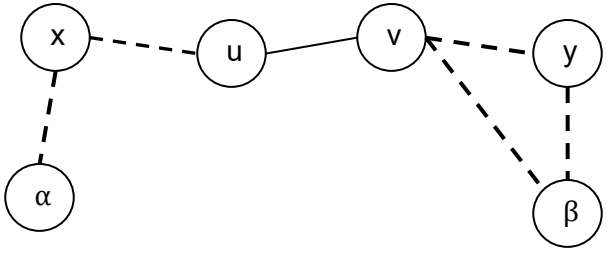


Exemplu rulare (CTC)

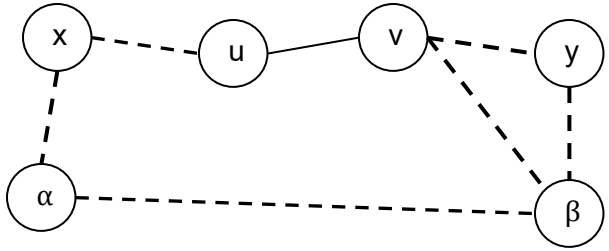


Punți

- Definiție: G = (V,E), graf neorientat și (u,v) ∈ E. (u,v) este punte în G ⇔ ∃ x,y ∈ V, x ≠ y, a.î. ∀ x..y conține muchia (u,v).



Orice drum x..y trece prin (u,v) =>(u,v) este punte



(u,v) nu este punte

Algoritm punți (I)

- Punți(G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** nod u (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - punte(u) =0; // înlocuiește: subarb(u) = 0; art(u) = 0;
  - **Pentru fiecare** nod u (u ∈ V)
    - **Dacă** c(u) e alb
      - Explorează(u)

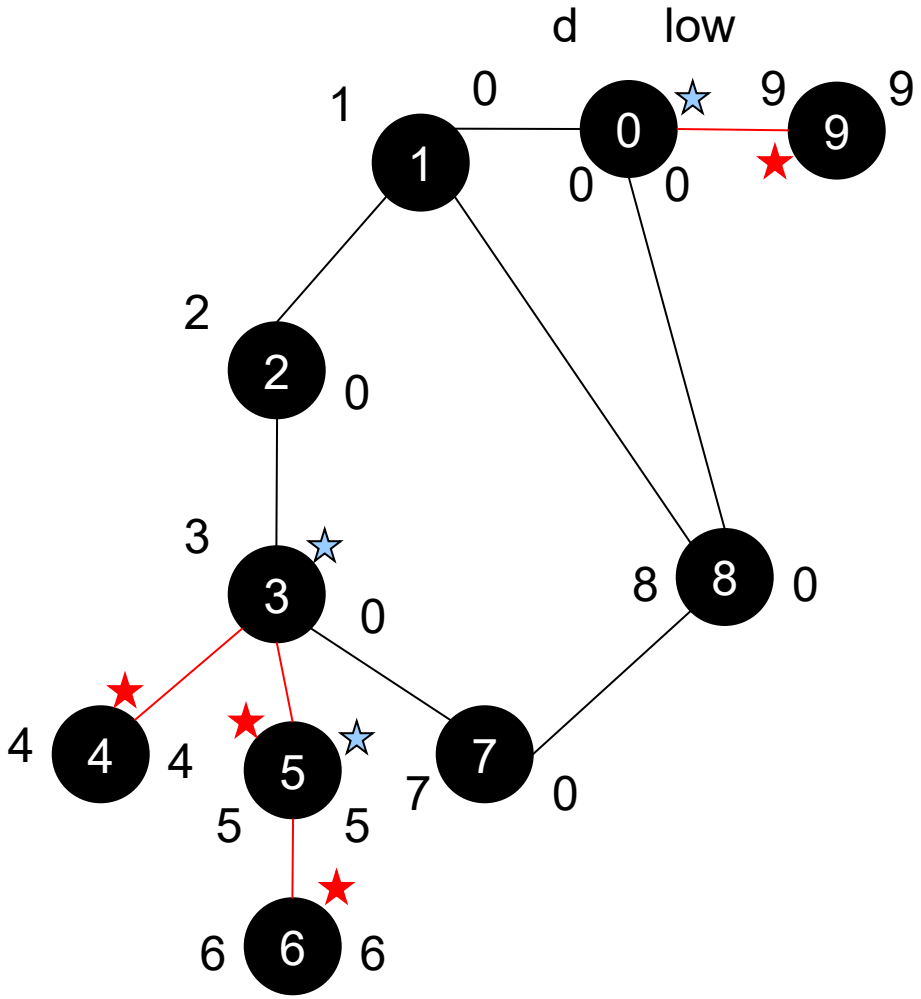
Algoritm punți (II)

low[u] = min {d(u), d(v) pentru toate muchiile înapoi (u,v), low(w) pentru toți fiii w ai lui u}

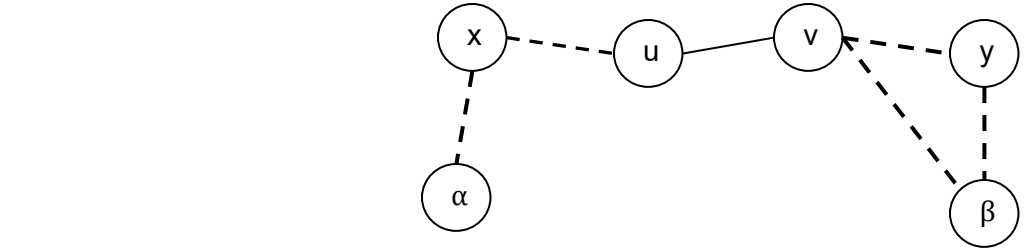
- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v (v ∈ succs(u))
    - **Dacă** c(v) e alb
      - p(v) = u; // se elimină: subarb(u)++;
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (low(v) > d(u)) punte(v) = 1;

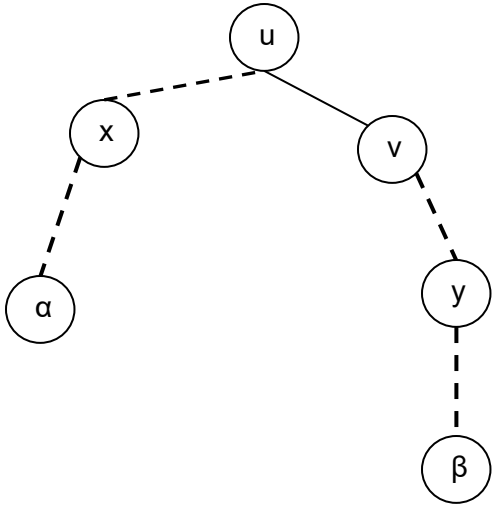
      // în loc de: **Dacă**(p(u) != null && low(v) ≥ d(u))
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

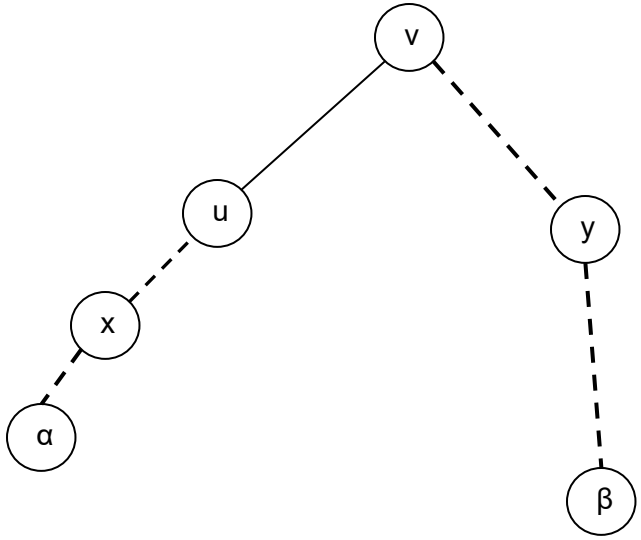
Exemplu rulare (Punți)



Exemplu







DFS din u; puntea este detectată în v

DFS din v; puntea este detectată în u

ÎNTREBĂRI?



Bibliografie

[1] Giumale – Introducere în Analiza Algoritmilor cap. 5.4, 5.4.1

[2] Cormen – Introducere în Algoritmi cap. Heap-uri binomiale (20), Heap-uri Fibonacci (21), Drumuri minime de sursă unica - primele 2 subcapitole (25.1 și 25.2)

[3] R. Sedgewick, K. Wayne - Algorithms and Data Structures Fall 2007 – Curs Princeton - http://www.cs.princeton.edu/~rs/AlgsDS07/06PriorityQueues.pdf

[4] Heap Fibonacci: https://www.cs.usfca.edu/~galles/JavascriptVisual/FibonacciHeap.html

[5] Fibonacci heap: http://www.growingwiththeweb.com/data-structures/fibonacci-heap/overview/